

Will We Eat the Rich If We Run Out of Cake?

Analysing the Cost of Living's Impact on Crime Rates for the City of Chicago

Brina DeWeese

Table of Contents

1. Introduction

- 1.1 Literature Review
- 1.2 Research Aims and Objectives

2. Data Processing and Preparation

- 2.1 Data Sets and Validation
 - 2.1.1 Cost of Living Validation
- 2.2 Data Import
- 2.3 Data Analysis
 - 2.3.1 Correlation
 - 2.3.2 Regression Analysis
 - 2.3.3 Random Forests and Gradient Boosting
 - 2.3.4 Forward Selection Linear Regression on Individual Crime Categories

3. Data Analysis Conclusions

4. Predictive Model

5. Limitations and Challenges

1.1 Literature Review

The idea that cost of living impacts crime rates is not new. Recent research, as of 2016, indicates that fluctuations in consumer prices over time strongly predict financially motivated crimes in the United States (Nunley et al., 2016 : Rosenfeld, 2016). However, the United States, when examined as a whole, consists of many cities and states with varying degrees of cost of living and crime. It is a common understanding that the costs of goods and crime rates vary significantly between cities in the United States. For example, on average, it is much cheaper to buy a home in Kansas City than it would be to buy that same home in New York City. The same variances can be seen in crime rates. The incidence of robbery in Houston is almost double compared to Seattle (Rosenfeld, 2018). These substantial disparities in crime rates and prices across different cities might lead to doubts about whether the correlation between crime and inflation observed on a national scale is equally applicable at the city level.

In their study, Rosenfeld (2018) attempted to replicate and extend prior national- level research by examining the link between financially motivated crimes and inflation across 17 cities in the United States, covering the period from 1960 to 2013. To do so, they employed a random coefficients linear panel model. Their study found that inflation had a notable impact on crime across all 17 cities. It was observed that an increase in inflation by one unit corresponds to an increase of about 96 acquisitive crimes for every 100,000 people. This finding suggests a clear link between heightened inflation rates and an uptick in financially motivated crimes like theft and burglary.

By examining 17 different cities in their study, Rosenfeld (2018) was able to note considerable differences in how inflation impacted crime rates from city to city. Chicago was one such city examined. In contrast to some other cities, Rosenfeld’s (2018) study revealed that in Chicago, the influence of inflation on the variation of acquisitive crime rates was not found to be statistically significant. This implies that, within the scope of the study’s data and methodology, fluctuations in Chicago’s inflation rates did not noticeably affect acquisitive crime occurrences, such as theft, burglary, or robbery. Chicago, according to this study, is an interesting case study. This case of Chicago suggests the need for further exploration into alternative crime types and diverse cost of living factors, and how they uniquely influence crime patterns in different urban contexts.

Field’s (1990) analysis provides insight into how cost of living factors affect various crime categories. Their paper show that different types of crimes respond differently to economic conditions, which could perhaps explain the variances between cities shown in Rosenfeld’s

6. Conclusions

7. Future Research

8. Theoretical Testing

- 8.1 Theoretical Data Model Conclusions

9. References

1. Introduction

Criminal behavior, with its myriad causes and complexities, has long been a subject of fascination and debate. From the tales of heroic figures like Aladdin and Robin Hood to the provocative moral question, 'Would you steal a loaf of bread to feed your family?', our cultural narratives often blur the lines between right and wrong. Throughout human history, the spectrum of criminal activity has spanned from rebellious revolutionaries to notorious gangs, and from legendary heroes to infamous villains. Even stories of royalty, such as the legendary queen famously fixated on cake, are woven into this narrative. In this intricate tapestry of human behavior, an essential question arises: What factors influence crime rates? This is not a new question. 2,600 years ago, Confucius identified that "Inequality and insecurity are more dangerous to the rulers than poverty" (Waley, A. (2011)). This notion can be seen repeated throughout history. The French Revolution was famously a result of socio-economic frustrations of the peasant class. At what point do socio-economic inequalities lead to criminal behaviors? Could cost of living information be used to keep a King’s head securely attached to his body? During the French Revolution, there were significant socio-economic disparities felt by the peasant class. Between 1741 and 1785, the real cost of living surged by 62% (Carteron, 2020). The years 1788 and 1789 saw significant crop failures, leading to a spike in bread prices alongside a decrease in wages. Specifically, in 1789, real wages plummeted by 25%, while the cost of bread soared by 88% (Carteron, 2020). Comparatively, in modern-day America, economic shifts over time reveal a similar pattern. According to Chuton (2023), in 1980, the annual cost of living was approximately 25,000 USD, and the average salary was 19,500 USD. Fast forward to 2022, and the cost of living had risen to 60,000 USD, a 140\% increase, while the average salary had grown to 55,000 USD (Chuton, 2023). This growth in salary, although significant, does not proportionally match the increase in the cost of living, mirroring the economic pressures similar to those observed during the French Revolution. So, do fluctuations in the cost of living lead to changes in crime rates, and if so, at what point will today’s American ‘peasants’ metaphorically ‘storm the Bastille’ if the proverbial cake becomes unaffordable?

(2018) study. In Field’s (1990) study, property crimes in England and Wales were found not to correlate significantly with unemployment rates; however, these crimes tend to increase alongside higher consumption levels, which often accompany economic growth. This trend suggests that the increased availability of goods and money could potentially lead to more opportunities for theft. Conversely, crimes against the person, such as sexual offenses and violence, show a different pattern, increasing with growing affluence and consumption. This could be linked to behavioral changes like spending more time outside the home, potentially elevating the risk of encountering such crimes (Field, 1990).

Field’s interpretation aligns with Routine Activity theory, which associates increases in affluence with more opportunities for certain types of crimes to occur (Fox, 2011). Given this, it may be pertinent to classify different crimes according to a category, such as ‘violent crimes’ or ‘economically motivated crimes’ when examining the impact that cost of living has on crime rates. Differences in correlations between cost of living and different crime categories may be an important understanding in future research.

Taking a different look at crime rates, Fox (2011) investigated the effects of an economic downturn and continued economic challenges in the UK on criminal activities in Greater Manchester. This research employed models of time allocation derived from Rational Choice theory, proposing that individuals balance the decision between lawful and unlawful activities considering the anticipated returns. Additionally, it used opportunity models that associate criminal behavior with the presence of potential targets for victimization. Their study acknowledged that areas with higher numbers of low-paid workers and greater poverty are more likely to have higher crime rates (Fox, 2011). They also found that despite the recession, crime rates in Greater Manchester have continued to decline; This included both violent and acquisitive crimes. The study indicated that the influence of the recession on crime rates might be moderated by the intricacies of the labor market and potentially subject to a delay in effect or a time lag. Notably, Fox (2011) highlighted the challenge in forecasting future crime trends, attributing this to the ambiguous results of prior studies and the intricate interaction of various economic factors.

1.12 Conclusion

From the broad national analyses in the United States, such as those by Nunley et al. (2016) and Rosenfeld (2016), to more localized examinations like Rosenfeld’s (2018) study across 17 U.S. cities, including Chicago, the evidence points to a nuanced interaction between economic factors and criminal behavior. These studies collectively reveal that while there is a general trend connecting inflation and financially motivated crimes, this relationship can vary significantly at the city level. Notably, in cities like Chicago, the

expected correlation between inflation and acquisitive crime rates is absent, suggesting the need for a more localized or differentiated approach in understanding and predicting crime trends in this city.

Field's (1990) work adds another layer to this understanding, highlighting how different types of crimes respond to economic conditions in varying ways. Their analysis, focusing on England and Wales, indicates a correlation between higher consumption levels and property crimes, while violent crimes show a different pattern, potentially influenced by lifestyle changes associated with affluence. This differentiation in crime types emphasizes the importance of considering specific categories of crime when examining the impact of cost of living.

Fox (2011) emphasizes the need for a more nuanced approach when examining crime rates and economic variables. The continued decline in crime rates despite economic challenges in Greater Manchester suggests that other factors, including labor market complexities and potential time lags, play a role in mediating the relationship between economic stressors and crime. Fox (2011) also notes the difficulty in forecasting crime rates based on economic challenges, which pose a hurdle for future research in this area.

The relationship between cost of living and crime rates is not straightforward and is influenced by a range of factors, including economic conditions, type of crime, and local context. Future research is needed to explore these dynamics, potentially employing more nuanced models and methodologies to unravel the complex interplay of economic factors and crime rates across different settings and crime categories.

1.2 Research Aims and Objectives

Research Aim:

To explore and understand the relationship between various cost of living factors and the rates of different crime categories within the city of Chicago, aiming to determine how specific economic elements influence diverse types of criminal activities in this urban setting.

Research Objectives:

- Objective 1: To examine the correlation between specific cost of living indicators unique to Chicago (including average monthly net salary after tax, mortgage interest rates, average rent prices, prices per square meter for property purchases, utility costs, restaurant meal prices, drink prices, grocery costs, and fun activity

- Threat, Burglary, Motor Vehicle Theft, Robbery: These are crimes typically committed with the intention of gaining property or money illegally.
- Deceptive Practice: Involves fraud or deception for financial gain.
 - Gambling, Prostitution: These are considered economically motivated as they are often undertaken for financial profit.
 - Arson: Arson can be economically motivated if it used for insurance fraud or another financial gain.

Violent Crimes:

- Battery, Assault, Criminal Sexual Assault, Homicide, Kidnapping, Intimidation, Sex Offense, Stalking: These are direct crimes against individuals, involving physical harm or the threat of harm.

Drug Related Crime:

- Narcotics, Other Narcotic Violation: These are directly related to the illegal trade, manufacture, or use of drugs (controlled substances).
 - Public Crime:
 - Liquor Law Violation, Public Peace Violation, Weapons Violation, Interference with Public Officer, Concealed Carry License Violation, Obscenity, Public Indecency: These are offenses that disrupt public order, safety, and morals.
 - Property Related Crime:
 - Criminal Damage, Criminal Trespass: These involve unlawful damage to or intrusion on someone else's property.

Vulnerable Populations related Crime:

- Offense Involving Children, Human Trafficking: These crimes are targeted at vulnerable groups, particularly children and victims of human trafficking, often involving exploitation or harm.
- Miscellaneous Crimes:
- Other Offense, Ritualism, Non-Criminal, Non-Criminal (Subject Specified), Non - Criminal: This is a diverse category encompassing various offenses that don't neatly fit into the other categories. 'Non-criminal' may include acts that are not legally crimes but were still recorded for documentation purposes.

These categorizations make logical sense, however, there could be arguments for certain crimes to be sorted in different categories. For example, Arson, if not related to fraud, might need to be sorted into property related crimes or even violent crimes. The Chicago Crimes Data Set does not specify motive for the crimes listed, so, for the

prices) and the incidence of various crime categories (such as drug-related crimes, economically motivated crimes, miscellaneous crimes, property-related crimes, public order crimes, violent crimes, and crimes against vulnerable populations).

This objective is designed to identify which economic factors have the most significant impact on each type of crime in Chicago.

- Objective 2: To create a predictive model focused on the city of Chicago that evaluates how fluctuations in cost of living may influence different crime rates. The model will aim to forecast changes in crime rates in response to predicted economic changes, providing valuable insights for city planners, policymakers, law enforcement, and community organizations in their efforts to develop effective crime prevention strategies and allocate resources more efficiently.

2. Data Processing and Preparation

2.1 Data Sets and Validation

The data set used for the regression and other various data analysis techniques is: small_crimes_and_costofliving.csv. This data set was created through combining elements of a Chicago Crimes Data set, Chicago Crimes: Crimes - 2001 to Present, and a web-scraped Chicago cost of living data set from Numbeo.

- The Chicago Crimes Data Set can be found here, <https://catalog.data.gov/dataset/crimes-2001-to-present>. However, please keep in mind that that Chicago Crimes data set found in that link was recently updated on December 9th, 2023. The updated version is missing copious amounts of data entries, in particular, entries in the years 2004-2019. A more accurate data set can be found in an archived version of the site with the same data set, updated on April 1st, 2023: <https://web.archive.org/web/20230403151625/https://catalog.data.gov/data/2001-to-present>. This discrepancy in the data sets caused significant issues and delays in regard to forecasting and regressing the data as well as causing difficulty when setting up theoretical tests using the years 2001-2022. Nonetheless, if the data set from April 1st, 2023 is used, then this research can be replicated.

Using the Primary Type column in the Chicago Crimes Data Set, crimes were categorized into the following categories:

- Economically Motivated Crimes:

purposes of this project it is assumed that arson is economically motivated. Of course, this needs to be taken into consideration when examining the reliability of the regression.

Later on, the Chicago crimes data set is manipulated again in order to get a count of the number of crimes committed each year in each of the previously mentioned categories. The crime rate information is then able to be used as a target for regression.

- Numbeo's Chicago Cost of Living Page, <https://www.numbeo.com/cost-of-living/city-history/in/Chicago>.

The Chicago Crimes data set was reduced to match the years represented in the Numbeo Cost of Living site, 2010-2022. Although it would be beneficial to utilize all of the Chicago Crimes data, the cost of living data was not accessible for the years 2001-2009 on Numbeo. However, for a portion of this research, in order to theoretically assess how additional information may impact the regression outcomes, the inflation rate was used to backfill for the missing years, 2001-2009; however, this may not be accurate as inflation can affect different cost of living categories differently throughout the years. This was simply done to use as a theoretical test set to increasing the size of the training set available.

On the same note, the inflation rate was also used to fill in any null values in the Numbeo Cost of Living Data. When web scraping numbeo, null values became prevalent in the produce data for information on groceries, such as the price of bananas. To fix this, I generated an inflation data set by web-scraping www.usinflationcalculator.com/inflation/current-inflation-rates/.

Using the df_inflation, I backfilled and forward-filled missing data using the following code:

```
inflation_dict = df_inflation1022.set_index('Year')
['Ave'].to_dict()
for column in df_buy.columns:
    if column != 'Year':
        for i in range(len(df_buy)):
            if pd.isna(df_buy[column].iloc[i]):
                found = False
                for j in range(i + 1, len(df_buy)):
                    if not pd.isna(df_buy[column].iloc[j]):
                        found = True
                        next_year = int(df_buy['Year'].iloc[j])
                        next_value = float(df_buy[column].iloc[j])
                        break
```

```

        if found:
            # Backward filling the missing values using the
inflation rates
            for k in range(j-1, i-1, -1):
                curr_year = int(df_buy['Year'].iloc[k])
                if curr_year < 2022:
                    next_year = int(df_buy['Year'].iloc[k +
1])
                    inflation_rate =
inflation_dict.get(next_year, 0) / 100
                    next_value /= (1 + inflation_rate)
                    df_buy.iat[k,
df_buy.columns.get_loc(column)] = next_value
                else: # If no future values are found, like for
year 2022, use the year before
                    for k in range(i - 1, -1, -1):
                        if not pd.isna(df_buy[column].iloc[k]):
                            last_known_value =

df_buy[column].iloc[k]

                        df_buy.iat[i,
df_buy.columns.get_loc(column)] = last_known_value
                        break

```

As stated previously, the inflation rate might not be the same across every sector of cost of living. So, there may be discrepancies in some of the calculated cost of living values. Knowing this, the accuracy of the cost of living data may not be 100%. However, using the inflation rate to backfill missing values is logical and the accuracy can be assessed by performing validation analysis on the cost of living data.

2.1.1 Cost of Living Validation

The produce information on Numbeo was the section with the majority of missing values that needed to be backfilled using the inflation rate. The column with the most missing data was the price of bananas, which had five values that had to be filled in using the inflation rate. To validate this filled in information, the bananas column can be examined and compared to the actual average price.

```

In [1]: import pandas as pd
df_allmarket = pd.read_csv('df_allmarket.csv')
banana_info = df_allmarket['Banana (1kg)'].unique()
print(banana_info)

```

```

Out[3]:
U.S. retail price of bananas 1995-2022  Unnamed: 1
17      2010.0      0.59
18      2011.0      0.60
19      2012.0      0.61
20      2013.0      0.59
21      2014.0      0.59
22      2015.0      0.58
23      2016.0      0.58
24      2017.0      0.56
25      2018.0      0.58
26      2019.0      0.57
27      2020.0      0.57
28      2021.0      0.62
29      2022.0      0.63

```

```

In [4]: # Multiply the data in column 'Unnamed: 1' by 2.20462
df_actual_banana_2010_2022.loc[:, 'Unnamed: 1'] = df_actual_banana_2010_2
df_actual_banana_2010_2022

```

```

Out[4]:
U.S. retail price of bananas 1995-2022  Unnamed: 1
17      2010.0      1.300726
18      2011.0      1.322772
19      2012.0      1.344818
20      2013.0      1.300726
21      2014.0      1.300726
22      2015.0      1.278680
23      2016.0      1.278680
24      2017.0      1.234587
25      2018.0      1.278680
26      2019.0      1.256633
27      2020.0      1.256633
28      2021.0      1.366864
29      2022.0      1.388911

```

With this actual banana data, it can be compared to the estimated data and can be examined using the margin of error. Of course, it should be noted that the actual banana data is price per pound for the whole united states, not specifically

```

<bound method Series.unique of 0      1.562966
1      1.612981
2      1.646854
3      1.671557
4      1.698302
5      1.700000
6      1.290000
7      1.770000
8      1.790000
9      1.460000
10     1.580000
11     1.570000
12     1.720000
Name: Banana (1kg), dtype: float64>

```

In the original Numbeo web scraped data, the years 2010 - 2014 for banana prices were null values. So, all of these values were backfilled using the inflation data. The filled in values are as follows:

- 2010
 - 1.562966
- 2011
 - 1.612981
- 2012
 - 1.646854
- 2013
 - 1.671557
- 2014
 - 1.698302

To compare, the data set for historic price of bananas in the USA was downloaded from Statista: <https://www.statista.com/statistics/236880/retail-price-of-bananas-in-the-united-states/>

```

In [2]: df_actual_banana = pd.read_csv('statistic_id236880_us-retail-price-of-ban
df_actual_banana_columns = df_actual_banana.columns
print(df_actual_banana_columns)
df_actual_banana['U.S. retail price of bananas 1995-2022'] = pd.to_nume
df_actual_banana_2010_2022 = df_actual_banana[df_actual_banana['U.S. reta
Index(['U.S. retail price of bananas 1995-2022', 'Unnamed: 1'], dtype='ob
ject')

```

```

In [3]: df_actual_banana_2010_2022
#the issue here is that this banana price is in in U.S. dollars per pound
# the conversion from pound to kg is Price per kilogram=Price per pound*2

```

Chicago. This could bring the price down, as Chicago is a large metropolitan city which typically have higher prices than in smaller towns or other states in general. With this in mind, a small margin of error, most likely with price skewed higher should be expected.

```

In [5]: data = {
'Years': [2010, 2011, 2012, 2013, 2014],
'Predicted Price': [1.562966, 1.612981, 1.646854, 1.671557, 1.698302],
'Actual Price': [1.300726, 1.322772, 1.344818, 1.300726, 1.300726]
}

df_realandpredicted_bananas = pd.DataFrame(data)

```

```

In [6]: df_realandpredicted_bananas

```

```

Out[6]:
  Years  Predicted Price  Actual Price
0  2010      1.562966      1.300726
1  2011      1.612981      1.322772
2  2012      1.646854      1.344818
3  2013      1.671557      1.300726
4  2014      1.698302      1.300726

```

The values are similar but not exact. However, as the actual values were an average for the entire United States, it is reasonable to assume that these values are skewed to be less than the price would be in a major metropolitan US city like Chicago. When comparing the predicted prices to the actual prices of bananas over the specified years, it's evident that the predicted prices tend to be higher than the actual prices. This suggests that the predictions may have overestimated the price of bananas. The difference between the predicted and actual prices, while not substantial, indicates a degree of variability in banana prices. Factors such as supply and demand fluctuations, weather conditions, and transportation costs can contribute to these variations. Therefore, while the predictions provide a reasonable estimate, real-world factors can influence the actual market prices, especially in urban areas with higher living costs like Chicago.

The process of backfilling and forward filling the cost of living information using the inflation rate is a reasonable and logical method for estimating values when specific data points are missing. However, it is important to acknowledge that there are some discrepancies between the actual values

and the estimated values obtained through this method. These variations may be attributed to various factors, including fluctuations in the real-world cost of living that are not fully accounted for by the inflation rate alone.

Additionally, it's worth noting that the cost of living data used in this analysis was sourced from Numbeo, a platform that relies on crowdsourced information. While crowdsourcing can provide valuable insights, it also introduces the possibility of bias or inaccuracies in the data. Despite these considerations, the estimated values obtained through backfilling and forward filling are still relatively close to the actual values. They serve as a valuable estimate and provide a useful reference point for this analysis.

2.2 Data Import

The crimes and cost of living data set is utilised here. However, the crime information in the data set is manipulated in order to get a crime rate for each year for each crime category. This was done by counting the occurrences of each type of crime in each year and then creating a new data frame with these crime counts and the cost of living averages information.

```
In [7]: # Libraries
import pandas as pd
import statsmodels.api as sm
from sklearn.linear_model import LinearRegression
from scipy import stats
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sb
from statsmodels.formula.api import mnlogit
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report,

In [8]: df_crimes= pd.read_csv('crimes_1022_small.csv')

In [9]: df_crimes.head()
```

	Year	Number of Drug-Related Crimes \	
0	2010	43404	
1	2011	38610	
2	2012	35495	
3	2013	34134	
4	2014	29128	
5	2015	23944	
6	2016	13337	
7	2017	11694	
8	2018	13591	
9	2019	15085	
10	2020	7498	
11	2021	5339	
12	2022	4738	
	Number of Economically Motivated Crimes	Number of Miscellaneous Crimes \	
0		152993	
1		151538	
2		145450	
3		130360	
4		114219	
5		108772	
6		120011	
7		121700	
8		117836	
9		109365	
10		87116	
11		84096	
12		110052	
	Number of Property-Related Crimes	Number of Public Crime Crime	
0		50055	881
1		45990	869
2		44071	875
3		38991	816
4		35338	787
5		35078	748
6		37324	631
7		35859	762
8		34732	865
9			

Out[9]:

	ID	Case Number	Date	Block	Primary Type	Description	Arrest	D
0	11037294	JA371270	2015-03-18 12:00:00	0000X W WACKER DR	DECEPTIVE PRACTICE	FINANCIAL IDENTITY THEFT OVER \$ 300	False	
1	11646293	JC213749	2018-12-20 15:00:00	023XX N LOCKWOOD AVE	DECEPTIVE PRACTICE	FINANCIAL IDENTITY THEFT \$300 AND UNDER	False	
2	11645836	JC212333	2016-05-01 00:25:00	055XX S ROCKWELL ST	DECEPTIVE PRACTICE	FINANCIAL IDENTITY THEFT OVER \$ 300	False	
3	11645959	JC211511	2018-12-20 16:00:00	045XX N ALBANY AVE	OTHER OFFENSE	TELEPHONE THREAT	False	
4	11645601	JC212935	2014-06-01 00:01:00	087XX S SANGAMON ST	DECEPTIVE PRACTICE	FINANCIAL IDENTITY THEFT OVER \$ 300	False	

Now, I need to create columns with the number of crimes for each crime types listed for each year. That way, I'll be able to examine crime rates instead of simply categorizing the data.

```
In [10]: crime_counts = df_crimes.pivot_table(index='Year', columns='Category',

In [11]: all_years = range(2010, 2023) # 2023 is not included, range is up to 2022
crime_counts = crime_counts.reindex(all_years, fill_value=0)

In [12]: crime_counts.reset_index(inplace=True)
crime_counts.columns = ['Year'] + ['Number of ' + str(col) + ' Crim

In [13]: print(crime_counts)
```

	Number of Violent Crimes	Number of Vulnerable Populations Crim	
0	90604		26
1	84529		24
2	82690		23
3	75346		24
4	69660		25
5	69428		24
6	73013		24
7	72494		24
8	74369		24
9	74238		24
10	63255		20
11	64793		19
12	65980		18
13			

An interesting observation here is that the crime rates dramatically drop off in the years 2020 and 2021. This, of course, can be attributed to the Covid19 pandemic. The low crime rates in these years may skew the outcome of the analysis.

```
In [14]: #crime_counts.to_csv('crime_counts.csv', index=False)

In [15]: cost_of_living_df= pd.read_csv('small_crimes_and_costofliving.csv')
cost_of_living_df.head()
```

Out[15]:

	Year	Average Monthly Net Salary (After Tax)	Mortgage Interest Rate in Percentages (%), Yearly, for 20 Years Fixed-Rate	Average Rent Price	Average per Square Meter Buy Price	Total Average Cost of all Utilities	Average Restauraunt Meal Price	Aver D P
0	2010	2750.0	5.63	1300.875	1628.04	211.33	19.333333	3.2
1	2010	2750.0	5.63	1300.875	1628.04	211.33	19.333333	3.2
2	2010	2750.0	5.63	1300.875	1628.04	211.33	19.333333	3.2
3	2010	2750.0	5.63	1300.875	1628.04	211.33	19.333333	3.2
4	2010	2750.0	5.63	1300.875	1628.04	211.33	19.333333	3.2

```
In [16]: cost_of_living_averages = cost_of_living_df.drop(['Primary Type',  
cost_of_living_averages.head()
```

Out[16]:

	Year	Average Monthly Net Salary (After Tax)	Mortgage Interest Rate in Percentages (%), Yearly, for 20 Years Fixed-Rate	Average Rent Price	Average per Square Meter Buy Price	Total Average Cost of all Utilities	Average Restauraunt Meal Price	Aver D P
0	2010	2750.0	5.63	1300.875	1628.04	211.33	19.333333	3.2
1	2010	2750.0	5.63	1300.875	1628.04	211.33	19.333333	3.2
2	2010	2750.0	5.63	1300.875	1628.04	211.33	19.333333	3.2
3	2010	2750.0	5.63	1300.875	1628.04	211.33	19.333333	3.2
4	2010	2750.0	5.63	1300.875	1628.04	211.33	19.333333	3.2

```
In [17]: #cost_of_living_averages.to_csv('cost_of_living_averages', index=False  
In [18]: col_and_cr= pd.merge(cost_of_living_averages, crime_counts, on='Ye  
col_and_cr_per_year = col_and_cr.drop_duplicates(subset='Year')  
In [19]: col_and_cr_per_year
```

Out[21]:

	Year	Average Monthly Net Salary (After Tax)	Mortgage Interest Rate in Percentages (%), Yearly, for 20 Years Fixed-Rate	Average Rent Price	Average per Square Meter Buy Price	Total Average Cost of all Utilities	Average Restauraunt Meal Price	Av
2010	2750.00	5.63	1300.8750	1628.040000	211.33	19.333333		
2011	2800.00	5.26	1574.4800	1749.135000	315.70	19.000000		
2012	3102.86	4.65	1725.1175	1636.115000	221.48	25.500000		
2013	3686.99	4.37	1864.2775	3301.919291	186.42	23.800000		
2014	3246.85	4.40	1819.5425	3354.750000	150.28	25.833333		
2015	3003.10	4.18	1880.9475	2609.535000	156.38	28.833333		
2016	3578.85	3.96	2006.4950	2531.125000	188.14	27.416667		
2017	3327.80	3.95	2025.5000	2574.580000	179.11	26.916667		
2018	4055.27	4.27	1963.5575	2939.860000	187.01	29.833333		
2019	4211.87	4.27	2275.2125	3191.220000	200.19	29.333333		
2020	4541.45	4.04	2240.2925	4238.065000	212.41	32.666667		
2021	5583.44	2.96	2081.5750	3011.205000	226.71	36.166667		
2022	4976.19	5.63	2563.7875	3011.205000	234.93	36.666667		

```
In [22]: data = {  
    'Year': [2010, 2011, 2012, 2013, 2014, 2015, 2016, 2017, 2018,  
    'Number of Drug-Related Crimes': [43404, 38610, 35495, 34134,  
    'Number of Economically Motivated Crimes': [152993, 151538, 14  
    'Number of Miscellaneous Crimes': [22013, 20206, 17531, 18058,  
    'Number of Property-Related Crimes': [50055, 45990, 44071, 389  
    'Number of Public Crime Crimes': [8817, 8694, 8758, 8161, 7876  
    'Number of Violent Crimes': [90604, 84529, 82690, 75346, 69660  
    'Number of Vulnerable Populations Crimes': [2631, 2430, 2330,  
    }  
  
    df5 = pd.DataFrame(data)  
  
    df5.set_index('Year', inplace=True)  
  
    df5['Total Number of Crimes'] = df5.sum(axis=1)  
  
    total_crime1 = df5[['Total Number of Crimes']]  
  
    total_crime1
```

Out[19]:

	Year	Average Monthly Net Salary (After Tax)	Mortgage Interest Rate in Percentages (%), Yearly, for 20 Years Fixed-Rate	Average Rent Price	Average per Square Meter Buy Price	Total Average Cost of all Utilities	Av Restu Meal
0	2010	2750.00	5.63	1300.8750	1628.040000	211.33	19.3
370519	2011	2800.00	5.26	1574.4800	1749.135000	315.70	19.0
722517	2012	3102.86	4.65	1725.1175	1636.115000	221.48	25.5
1058846	2013	3686.99	4.37	1864.2775	3301.919291	186.42	23.8
1366391	2014	3246.85	4.40	1819.5425	3354.750000	150.28	25.8
1642196	2015	3003.10	4.18	1880.9475	2609.535000	156.38	28.8
1907007	2016	3578.85	3.96	2006.4950	2531.125000	188.14	27.4
2176860	2017	3327.80	3.95	2025.5000	2574.580000	179.11	26.9
2445977	2018	4055.27	4.27	1963.5575	2939.860000	187.01	29.8
2714909	2019	4211.87	4.27	2275.2125	3191.220000	200.19	29.3
2976301	2020	4541.45	4.04	2240.2925	4238.065000	212.41	32.6
3188568	2021	5583.44	2.96	2081.5750	3011.205000	226.71	36.1
3397542	2022	4976.19	5.63	2563.7875	3011.205000	234.93	36.6

```
In [20]: #col_and_cr_per_year.to_csv('col_and_cr_per_year.csv', index=False  
In [21]: col_and_cr_per_year = pd.read_csv('col_and_cr_per_year.csv')  
col_and_cr_per_year.set_index('Year', inplace=True)  
col_and_cr_per_year
```

Out[22]:

	Total Number of Crimes
Year	
2010	370517
2011	351997
2012	336325
2013	307542
2014	275796
2015	264801
2016	269839
2017	269111
2018	268923
2019	261366
2020	212235
2021	208917
2022	239004

```
In [23]: cost_of_living_averages = pd.read_csv('cost_of_living_averages')  
total_crime = pd.merge(cost_of_living_averages, total_crime1, left  
total_crime.set_index('Year', inplace=True)  
total_crime
```

Out[23]:

	Average Monthly Net Salary (After Tax)	Mortgage Interest Rate in Percentages (%), Yearly, for 20 Years Fixed-Rate	Average Rent Price	Average per Square Meter Buy Price	Total Average Cost of all Utilities	Average Resturaunt Meal Price	Average Drinkl Price
Year							
2010	2750.0	5.63	1300.875	1628.04	211.33	19.333333	3.2021
2010	2750.0	5.63	1300.875	1628.04	211.33	19.333333	3.2021
2010	2750.0	5.63	1300.875	1628.04	211.33	19.333333	3.2021
2010	2750.0	5.63	1300.875	1628.04	211.33	19.333333	3.2021
2010	2750.0	5.63	1300.875	1628.04	211.33	19.333333	3.2021
2010	2750.0	5.63	1300.875	1628.04	211.33	19.333333	3.2021
2010	2750.0	5.63	1300.875	1628.04	211.33	19.333333	3.2021
2010	2750.0	5.63	1300.875	1628.04	211.33	19.333333	3.2021
2010	2750.0	5.63	1300.875	1628.04	211.33	19.333333	3.2021
2010	2750.0	5.63	1300.875	1628.04	211.33	19.333333	3.2021
2010	2750.0	5.63	1300.875	1628.04	211.33	19.333333	3.2021
2010	2750.0	5.63	1300.875	1628.04	211.33	19.333333	3.2021
2010	2750.0	5.63	1300.875	1628.04	211.33	19.333333	3.2021
2010	2750.0	5.63	1300.875	1628.04	211.33	19.333333	3.2021
2010	2750.0	5.63	1300.875	1628.04	211.33	19.333333	3.2021

```
In [24]: #total_crime.to_csv('total_crime.csv', index=True)
```

2.3 Data Analysis

With the col_and_cr_per_year, which contains both crime statistics per crime type and cost of living data per year, as well as total_crime which has the total number of crimes per year and the cost of living data, there are several machine learning techniques I can apply for various purposes, such as:

- 2.3.1 Correlation Analysis:
 - Objective: Explore the relationships between different types of crimes and various cost of living factors.
 - Implementation: Calculate correlation coefficients to understand the strength and direction of relationships between variables.

```
In [26]: crime_columns = ['Number of Drug-Related Crimes', 'Number of Economic Crimes', 'Number of Miscellaneous Crimes', 'Number of Public Crime Crimes', 'Number of Violent Crimes', 'Number of Vulnerable Populations Crimes']

correlation_matrix = col_and_cr_per_year.corr()

cost_of_living_columns = ['Average Monthly Net Salary (After Tax)', 'Mortgage Interest Rate in Percentages (%)', 'Average Rent Price', 'Average per Square Meter Buy Price', 'Total Average Cost of all Utilities', 'Average Restaurant Meal Price', 'Average Drink Price', 'Total Average Cost of all Groceries']

crime_correlations = correlation_matrix[crime_columns].loc[cost_of_living_columns]
print(crime_correlations)
```

- 2.3.2 Regression Analysis:
 - Objective: Predict crime rates based on cost of living factors.
 - Techniques: Linear Regression, k-nearest neighbors (KNN)
 - Implementation: Use cost of living variables as features and crime counts as target variables. This can help to understand how different cost of living factors might correlate with crime rates.
- 2.3.3 Random Forests and Gradient Boosting:
 - Objective: Predict crime rates based on cost of living and crime data.
 - Techniques: Random Forests and Gradient Boosting.
 - Implementation: Both Random Forests and Gradient Boosting are versatile techniques that can be used for both regression and classification tasks.
- 2.3.4 Forward Selection Linear Regression on Individual Crime Categories:
 - Objective: Reduce the dimensionality of the data and identify key components for individual crime categories.
 - Implementation: Forward selection linear regression is used to build a model by iteratively adding the most significant predictor variables. This approach helps reduce the number of variables and identifies the key components driving crime rates for each individual crime category.

2.3.1 Correlation

- Objective: Explore the relationships between different types of crimes and various cost of living factors. - Implementation: Calculate correlation coefficients to understand the strength and direction of relationships between variables. Using col_and_cr_per_year and total_crime correlations can be calculated to determine the correlation between cost of living factors and crime types, as well as total crime numbers.

```
In [25]: col_and_cr_per_year = pd.read_csv('col_and_cr_per_year.csv')
total_crime = pd.read_csv('total_crime.csv')
```

ug-Related Crimes \	Number of Dr
Average Monthly Net Salary (After Tax)	
-0.809935	
Mortgage Interest Rate in Percentages (%), Year...	
0.518124	
Average Rent Price	
-0.881709	
Average per Square Meter Buy Price	
-0.622847	
Total Average Cost of all Utilities	
0.162331	
Average Resturaunt Meal Price	
-0.903083	
Average Drink Price	
-0.872912	
Total Average Cost of all Groceries	
-0.737801	
Average Fun Activity Price	
-0.184317	
Number of Ec	
onomically Motivated Crimes \	
Average Monthly Net Salary (After Tax)	
-0.792563	
Mortgage Interest Rate in Percentages (%), Year...	
0.675036	
Average Rent Price	
-0.747597	
Average per Square Meter Buy Price	
-0.815061	
Total Average Cost of all Utilities	
0.324817	
Average Resturaunt Meal Price	
-0.872056	
Average Drink Price	
-0.681519	
Total Average Cost of all Groceries	
-0.541766	
Average Fun Activity Price	
-0.083169	
Number of Mi	
scellaneous Crimes \	
Average Monthly Net Salary (After Tax)	
-0.826459	
Mortgage Interest Rate in Percentages (%), Year...	
0.533290	
Average Rent Price	
-0.837675	
Average per Square Meter Buy Price	
-0.767626	
Total Average Cost of all Utilities	
0.124734	
Average Resturaunt Meal Price	
-0.901617	
Average Drink Price	
-0.757251	
Total Average Cost of all Groceries	
-0.549984	
Average Fun Activity Price	
-0.185700	
Number of Pr	

operty-Related Crimes \

Average Monthly Net Salary (After Tax)

-0.809934

Mortgage Interest Rate in Percentages (%), Year...

0.622552

Average Rent Price

-0.861114

Average per Square Meter Buy Price

-0.831473

Total Average Cost of all Utilities

0.314373

Average Restauraunt Meal Price

-0.906446

Average Drink Price

-0.752343

Total Average Cost of all Groceries

-0.614355

Average Fun Activity Price

-0.143772

blic Crime Crimes \

Average Monthly Net Salary (After Tax)

0.675407

Mortgage Interest Rate in Percentages (%), Year...

0.064902

Average Rent Price

0.426324

Average per Square Meter Buy Price

0.364041

Total Average Cost of all Utilities

0.425923

Average Restauraunt Meal Price

0.505907

Average Drink Price

0.469463

Total Average Cost of all Groceries

0.296883

Average Fun Activity Price

0.008603

olent Crimes \

Average Monthly Net Salary (After Tax)

-0.725172

Mortgage Interest Rate in Percentages (%), Year...

0.578402

Average Rent Price

-0.818438

Average per Square Meter Buy Price

-0.824169

Total Average Cost of all Utilities

0.342341

Average Restauraunt Meal Price

-0.864463

Average Drink Price

-0.698889

Total Average Cost of all Groceries

-0.574075

Average Fun Activity Price

-0.126654

Inerable Populations Crimes

- Average Drink Price (-0.872912):
- Economically Motivated Crimes Correlations
 - Mortgage Interest Rate in Percentages (%), Yearly, for 20 Years Fixed-Rate (0.675036):
 - Total Average Cost of all Groceries (-0.541766):
 - Average Drink Price (-0.681519):
- Miscellaneous Crimes Correlations
 - Mortgage Interest Rate in Percentages (%), Yearly, for 20 Years Fixed-Rate (0.533290):
 - Total Average Cost of all Groceries (-0.549984):
 - Average Drink Price (-0.757251):
- Property-Related Crimes Correlations
 - Mortgage Interest Rate in Percentages (%), Yearly, for 20 Years Fixed-Rate (0.622552):
 - Total Average Cost of all Groceries (-0.614355):
 - Average Drink Price (-0.752343):
- Public Crime Correlations
 - Average Monthly Net Salary (After Tax) (0.675407):
- Violent Crimes Correlations
 - Mortgage Interest Rate in Percentages (%), Yearly, for 20 Years Fixed-Rate (0.578402):
 - Average Rent Price (-0.818438):
 - Average per Square Meter Buy Price (-0.824169):
 - Total Average Cost of all Utilities (0.342341):
- Vulnerable Populations Crimes Correlations
 - Average Monthly Net Salary (After Tax) (-0.840691):

Average Monthly Net Salary (After Tax)

-0.840691

Mortgage Interest Rate in Percentages (%), Year...

0.192829

Average Rent Price

-0.691115

Average per Square Meter Buy Price

-0.423078

Total Average Cost of all Utilities

-0.334915

Average Restauraunt Meal Price

-0.812262

Average Drink Price

-0.723984

Total Average Cost of all Groceries

-0.467845

Average Fun Activity Price

-0.192964

Correlations with Crime Types

Strong Correlations

- Drug-Related Crimes Correlations
 - Average Rent Price (-0.881709):
 - Average Restauraunt Meal Price (-0.903003):
- Economically Motivated Crimes Correlations
 - Average Rent Price (-0.747597):
 - Average Restauraunt Meal Price (-0.872056):
- Miscellaneous Crimes Correlations
 - Average Rent Price (-0.837675):
 - Average Restauraunt Meal Price (-0.901617):
- Property-Related Crimes Correlations
 - Average Rent Price (-0.861114):
 - Average Restauraunt Meal Price (-0.906446):

Moderate Correlations

- Drug-Related Crimes Correlations
 - Average per Square Meter Buy Price (-0.622047):
 - Total Average Cost of all Groceries (-0.737001):

- Mortgage Interest Rate in Percentages (%), Yearly, for 20 Years Fixed-Rate (0.192829):
- Average Rent Price (-0.691115):
- Average Restauraunt Meal Price (-0.812262):
- Average Drink Price (-0.723984):

Weak Correlations

- Drug-Related Crimes Correlations
 - Total Average Cost of all Utilities (0.162331):
 - Average Fun Activity Price (-0.184317):
- Economically Motivated Crimes Correlations
 - Average Fun Activity Price (-0.083169):
- Miscellaneous Crimes Correlations
 - Total Average Cost of all Utilities (0.124734):
 - Average Fun Activity Price (-0.185700):
- Property-Related Crimes Correlations
 - Total Average Cost of all Utilities (0.314373):
 - Average Fun Activity Price (-0.143772):
- Public Crime Crimes Correlations
 - Mortgage Interest Rate in Percentages (%), Yearly, for 20 Years Fixed-Rate (0.064902):
 - Average Fun Activity Price (0.008603):

Key Takeaways: Correlated with a Decrease in Crime Rates

- Average Rent Price: As rent prices decrease, crime rates generally decrease. This is observed in various categories of crimes, including drug-related, economically motivated, miscellaneous, property-related, and even violent crimes.

- **Average Restaurant Meal Price:** Lower restaurant meal prices are correlated with lower crime rates. This correlation is strong across different crime categories, such as drug-related, economically motivated, miscellaneous, property-related, and violent crimes.
- **Average Drink Price:** Similarly, lower drink prices are associated with lower crime rates. This correlation holds true for various crime categories, including drug-related, economically motivated, miscellaneous, and property-related crimes.
- **Total Average Cost of all Groceries:** When the cost of groceries is lower, crime rates tend to be lower as well. This correlation is observed in drug-related, economically motivated, miscellaneous, and property-related crimes. **Policy Implications:** If these correlations are found to be causally significant through further analysis, they could inform policy decisions. For example, improving economic conditions might be a strategy to reduce crime rates.
- **Average Monthly Net Salary (After Tax):** Higher average monthly net salaries are correlated with lower crime rates. This negative correlation is observed in various crime categories, including drug-related, economically motivated, miscellaneous, property-related, public crime, violent crimes, and crimes against vulnerable populations.
- **Mortgage Interest Rate in Percentages (%) (Yearly):** Higher mortgage interest rates are associated with lower crime rates. This negative correlation is seen in drug-related, economically motivated, and property-related crimes.

```
In [27]: crime_columns2 = ['Total Crime Numbers']
correlation_matrix2 = total_crime.corr()

cost_of_living_columns2 = ['Average Monthly Net Salary (After Tax)',
'Mortgage Interest Rate in Percentage',
'Average Rent Price', 'Average per Square Meter Buy Price',
'Total Average Cost of all Utilities',
'Average Restaurant Meal Price', 'Average Drink Price',
'Total Average Cost of all Groceries']

crime_correlations2 = correlation_matrix2[crime_columns2].loc[cost_of_living_columns2]
print(crime_correlations2)
```

- **Average per Square Meter Buy Price (-0.795011):**
 - There is a moderate negative correlation between the average per square meter buy price and total crime numbers. Higher property prices per square meter are associated with lower total crime numbers.
- **Total Average Cost of all Groceries (-0.630280):**
 - Moderate negative correlation is found between the total average cost of all groceries and total crime numbers. Areas with expensive groceries tend to have slightly lower total crime numbers.

Weak Correlations

- **Average Monthly Net Salary (After Tax) (-0.814506):**
 - There is a weak negative correlation between the average monthly net salary (after tax) and total crime numbers. As income levels rise, total crime numbers tend to decrease, but the correlation is not very strong.
- **Average Fun Activity Price (-0.135335):**
 - The correlation between average fun activity prices and total crime numbers is weak. Areas with higher fun activity prices may have slightly lower total crime numbers, but the influence is limited.

Key Takeaways: Correlated with a Decrease in Crime Rates

- **Average Restaurant Meal Price:** As the price of restaurant meals increases, crime rates generally decrease. This strong negative correlation is observed across different crime categories.
- **Average Rent Price:** Higher rent prices are strongly correlated with lower crime rates. This relationship holds true for various crime categories, indicating a consistent negative correlation.
- **Average Monthly Net Salary (After Tax):** Areas with higher average monthly net salaries tend to have lower crime rates. This strong negative correlation is observed across multiple crime categories.

Numbers	Total Crime
Average Monthly Net Salary (After Tax)	-
0.814506	
Mortgage Interest Rate in Percentages (%), Yearly	-
0.637550	
Average Rent Price	-
0.842517	
Average per Square Meter Buy Price	-
0.795011	
Total Average Cost of all Utilities	-
0.296999	
Average Restaurant Meal Price	-
0.917390	
Average Drink Price	-
0.773076	
Total Average Cost of all Groceries	-
0.630280	
Average Fun Activity Price	-
0.135335	

Correlation With Total Number of Crimes

Strong Correlations

- **Average Restaurant Meal Price (-0.917390):**
 - There is a strong negative correlation between the average restaurant meal price and total crime numbers. As restaurant meal prices increase, total crime numbers tend to decrease significantly.
- **Average Rent Price (-0.842517):**
 - Expensive rent prices are strongly correlated with lower total crime numbers. #### Moderate Correlations
- **Mortgage Interest Rate in Percentages (%) (0.637550):**
 - There is a moderate positive correlation between mortgage interest rates and total crime numbers. Higher mortgage rates may be associated with slightly higher total crime numbers.
- **Average Drink Price (-0.773076):**
 - Moderate negative correlation exists between average drink prices and total crime numbers. Lower drink prices are associated with slightly higher total crime numbers.
- **Total Average Cost of all Utilities (0.296999):**
 - A moderate positive correlation is observed between the total average cost of all utilities and total crime numbers. Higher utility costs tend to have slightly higher total crime numbers.

- **Average Drink Price:** Lower drink prices are correlated with lower crime rates. This strong negative correlation holds true for various crime categories.
- **Total Average Cost of all Groceries:** When the cost of groceries is lower, crime rates tend to be lower as well. This strong negative correlation is observed in multiple crime categories.

2.3.2 Regression Analysis

- Objective: Predict crime rates based on cost of living factors.

Techniques: Linear Regression, k-nearest neighbors (KNN) -

Implementation: Use cost of living variables as features and crime counts as target variables. This can help to understand how different cost of living factors might correlate with crime rates.

K Nearest Neighbor

```
In [28]: import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsRegressor
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error

col_and_cr_per_year.set_index('Year', inplace=True)

features = col_and_cr_per_year[['Average Monthly Net Salary (After Tax)',
'Mortgage Interest Rate in Percentages (%)',
'Average Rent Price',
'Average per Square Meter Buy Price',
'Total Average Cost of all Utilities',
'Average Restaurant Meal Price',
'Average Drink Price',
'Total Average Cost of all Groceries',
'Average Fun Activity Price']]

targets = col_and_cr_per_year[['Number of Drug-Related Crimes',
'Number of Economically Motivated Crimes',
'Number of Miscellaneous Crimes',
'Number of Property-Related Crimes',
'Number of Public Crime Crimes',
'Number of Violent Crimes',
'Number of Vulnerable Populations']]

train_indices = col_and_cr_per_year.index <= 2020
test_indices = col_and_cr_per_year.index.isin([2021, 2022])

train_int_indices = col_and_cr_per_year.index[train_indices]
test_int_indices = col_and_cr_per_year.index[test_indices]
```



```

train_positions = [col_and_cr_per_year.index.get_loc(ind) for ind in train_indices]
test_positions = [col_and_cr_per_year.index.get_loc(ind) for ind in test_indices]

# Standardize features
scaler = StandardScaler()
train_features_scaled = scaler.fit_transform(features.loc[train_indices])
test_features_scaled = scaler.transform(features.loc[test_indices])

print("Train features shape:", train_features_scaled.shape)
print("Test features shape:", test_features_scaled.shape)

for target_col in targets.columns:
    train_target = targets.loc[train_indices, target_col]
    test_target = targets.loc[test_indices, target_col]

    knn = KNeighborsRegressor(n_neighbors=3)
    knn.fit(train_features_scaled, train_target)

    # MSE's
    predictions = knn.predict(test_features_scaled)
    mse = mean_squared_error(test_target, predictions)
    print(f'Test MSE for {target_col}: {mse}')

```

```

Train features shape: (11, 9)
Test features shape: (2, 9)
Test MSE for Number of Drug-Related Crimes: 49363680.5
Test MSE for Number of Economically Motivated Crimes: 2276928
20.1111104
Test MSE for Number of Miscellaneous Crimes: 1802980.0
Test MSE for Number of Property-Related Crimes: 8181537.6111
11175
Test MSE for Number of Public Crime Crimes: 299028.9444444451
Test MSE for Number of Violent Crimes: 27748742.944444496
Test MSE for Number of Vulnerable Populations Crimes: 152782.
2777777764

```

```

In [29]: mse_data = {
    'KNN Model': [
        49363680.5,
        227692820.1111104,
        1802980.0,
        8181537.611111175,
        299028.9444444451,
        27748742.944444496,
        152782.2777777764
    ]
}

mse_df = pd.DataFrame(mse_data, columns=['KNN Model'])

mse_df

```

- Drug-Related Crimes: The model predicts drug-related crimes with relatively low accuracy, as indicated by a Test Mean Squared Error (MSE) of approximately 49,363,680.5.
- Economically Motivated Crimes: Predictions for economically motivated crimes have a high Test MSE of about 227,692,820.11, suggesting that the model's performance in this category is less accurate.
- Miscellaneous Crimes: The model performs reasonably well in predicting miscellaneous crimes, with a Test MSE of 1,802,980.0, indicating relatively accurate predictions.
- Property-Related Crimes: Predictions for property-related crimes have a moderately high Test MSE of approximately 8,181,537.61, indicating room for improvement in accuracy.
- Public Crimes: The model predicts public crimes with relatively low error, as shown by a Test MSE of 299,028.94, indicating good predictive performance.
- Violent Crimes: Predictions for violent crimes have a high Test MSE of about 27,748,742.94, suggesting less accuracy in this category.
- Crimes Against Vulnerable Populations: The model performs well in predicting crimes against vulnerable populations, with a Test MSE of approximately 152,782.28, indicating accurate predictions.

The KNN model's predictive performance varies across different crime categories. It performs well for public crimes and crimes against vulnerable populations but struggles with drug-related and economically motivated crimes. There is also room for improvement in predicting property-related and violent crimes.

```

In [31]: total_crime.set_index('Year', inplace=True)

features = total_crime[['Average Monthly Net Salary (After Tax)',
                        'Mortgage Interest Rate in Percentages',
                        'Average Rent Price',
                        'Average per Square Meter Building Cost',
                        'Total Average Cost of all Urban Households',
                        'Average Restaurant Meal Price',
                        'Average Drink Price',

```

```

Out[29]:      KNN Model
0  4.936368e+07
1  2.276928e+08
2  1.802980e+06
3  8.181538e+06
4  2.990289e+05
5  2.774874e+07
6  1.527823e+05

```

```

In [30]: crime_categories = [
    'Number of Drug-Related Crimes',
    'Number of Economically Motivated Crimes',
    'Number of Miscellaneous Crimes',
    'Number of Property-Related Crimes',
    'Number of Public Crime Crimes',
    'Number of Violent Crimes',
    'Number of Vulnerable Populations Crimes'
]

mse_df.insert(0, 'crime category', crime_categories)

mse_df

```

```

Out[30]:      crime category      KNN Model
0      Number of Drug-Related Crimes  4.936368e+07
1  Number of Economically Motivated Crimes  2.276928e+08
2      Number of Miscellaneous Crimes  1.802980e+06
3      Number of Property-Related Crimes  8.181538e+06
4      Number of Public Crime Crimes  2.990289e+05
5      Number of Violent Crimes  2.774874e+07
6  Number of Vulnerable Populations Crimes  1.527823e+05

```

```

- Test MSE for Number of Drug-Related Crimes:
49363680.5
- Test MSE for Number of Economically Motivated
Crimes: 227692820.1111104
- Test MSE for Number of Miscellaneous Crimes:
1802980.0
- Test MSE for Number of Property-Related Crimes:
8181537.611111175
- Test MSE for Number of Public Crime Crimes:
299028.9444444451
- Test MSE for Number of Violent Crimes:
27748742.944444496
- Test MSE for Number of Vulnerable Populations
Crimes: 152782.2777777764

```

```

# Total Average Cost of all Urban Households
# Average Fun Activity Price

targets = total_crime[['Total Crime Numbers']]

train_indices = total_crime.index <= 2020
test_indices = total_crime.index.isin([2021, 2022])

train_int_indices = total_crime.index[train_indices]
test_int_indices = total_crime.index[test_indices]

train_positions = [total_crime.index.get_loc(ind) for ind in train_indices]
test_positions = [total_crime.index.get_loc(ind) for ind in test_indices]

# Standardize features
scaler = StandardScaler()
train_features_scaled = scaler.fit_transform(features.loc[train_indices])
test_features_scaled = scaler.transform(features.loc[test_indices])

print("Train features shape:", train_features_scaled.shape)
print("Test features shape:", test_features_scaled.shape)

for target_col in targets.columns:
    train_target = targets.loc[train_indices, target_col]
    test_target = targets.loc[test_indices, target_col]

    knn = KNeighborsRegressor(n_neighbors=3)
    knn.fit(train_features_scaled, train_target)

    # MSE's
    predictions = knn.predict(test_features_scaled)
    mse = mean_squared_error(test_target, predictions)
    print(f'Test MSE for {target_col}: {mse}')

Train features shape: (11, 9)
Test features shape: (2, 9)
Test MSE for Total Crime Numbers: 780791648.5

```

```

In [32]: mse_total_crime_data = {
    'KNN Model': [780791648.5]
}

# DataFrame for MSE_Total_Crime
mse_total_crime_df = pd.DataFrame(mse_total_crime_data, columns=['KNN Model'])

mse_total_crime_df

```

```

Out[32]:      KNN Model
0  780791648.5

```

- Test MSE for Total Crime Numbers: 780,791,648.5

Such a high MSE shows that there is not much predictive power in this model at all.

Bootstrapping and Bagging

```
In [33]: import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsRegressor
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error
from sklearn.ensemble import BaggingRegressor

features = col_and_cr_per_year[['Average Monthly Net Salary',
                                'Mortgage Interest Rate in',
                                'Average Rent Price',
                                'Average per Square Meter B',
                                'Total Average Cost of all',
                                'Average Restaurent Meal Pr',
                                'Average Drink Price',
                                'Total Average Cost of all',
                                'Average Fun Activity Price']

targets = col_and_cr_per_year[['Number of Drug-Related Crim',
                                'Number of Economically Moti',
                                'Number of Miscellaneous Cri',
                                'Number of Property-Related',
                                'Number of Public Crime Crim',
                                'Number of Violent Crimes',
                                'Number of Vulnerable Popula']

train_indices = col_and_cr_per_year.index <= 2020
test_indices = col_and_cr_per_year.index.isin([2021, 2022])

train_features = features[train_indices]
test_features = features[test_indices]

scaler = StandardScaler()
train_features_scaled = scaler.fit_transform(train_features)
test_features_scaled = scaler.transform(test_features)

for target_col in targets.columns:
    train_target = targets.loc[train_indices, target_col]
    test_target = targets.loc[test_indices, target_col]

    knn = KNeighborsRegressor(n_neighbors=3)

    # Bagging model
    bagging_model = BaggingRegressor(estimator=knn, n_estim

    bagging_model.fit(train_features_scaled, train_target)

    # Predicting and evaluating
    predictions = bagging_model.predict(test_features_scale
    mse = mean_squared_error(test_target, predictions)
    print(f'Test MSE for {target_col} (Bagging): {mse}')
```

- Test MSE for Number of Vulnerable Populations Crimes (Bagging): 161602.7744395553

Bootstrapping and bagging had varying impacts on the predictive accuracy of the KNN model across the different crime categories. While it improved predictive accuracy for some crime types (miscellaneous and vulnerable populations crimes), it still exhibited substantial prediction errors for others (economically motivated and violent crimes).

Linear Regression

```
In [35]: import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
import numpy as np
```

```
In [36]: features = col_and_cr_per_year[['Average Monthly Net Salary',
                                'Mortgage Interest Rate in',
                                'Average Rent Price',
                                'Average per Square Meter B',
                                'Total Average Cost of all',
                                'Average Restaurent Meal Pr',
                                'Average Drink Price',
                                'Total Average Cost of all',
                                'Average Fun Activity Price']

targets = col_and_cr_per_year[['Number of Drug-Related Crim',
                                'Number of Economically Moti',
                                'Number of Miscellaneous Cri',
                                'Number of Property-Related',
                                'Number of Public Crime Crim',
                                'Number of Violent Crimes',
                                'Number of Vulnerable Popula']
```

```
In [37]: train_set = col_and_cr_per_year.loc[2010:2020]

test_set = col_and_cr_per_year.loc[2021:2022]
```

```
In [38]: # Features for training and testing sets
features_train = train_set[features.columns]
features_test = test_set[features.columns]

# Targets for training and testing sets
targets_train = train_set[targets.columns]
targets_test = test_set[targets.columns]
```

```
In [39]: model = LinearRegression()
model.fit(features_train, targets_train)
```

```
Out[39]: ▾ LinearRegression
LinearRegression()
```

Test MSE for Number of Drug-Related Crimes (Bagging): 53512.126.7754945
Test MSE for Number of Economically Motivated Crimes (Bagging): 234356851.77510616
Test MSE for Number of Miscellaneous Crimes (Bagging): 1990627.151148444
Test MSE for Number of Property-Related Crimes (Bagging): 10239209.225949286
Test MSE for Number of Public Crime Crimes (Bagging): 851688.4350869209
Test MSE for Number of Violent Crimes (Bagging): 29064470.780335486
Test MSE for Number of Vulnerable Populations Crimes (Bagging): 179128.52420775784

```
In [34]: bagging_mse_values = [
51269892.82790248,
233712470.80579707,
2606274.6196996197,
11541921.9374055,
1010941.5455044294,
26176073.18503278,
161602.7744395553
]

mse_df['bootstrapping and bagging KNN'] = bagging_mse_value
mse_df
```

	crime category	KNN Model	bootstrapping and bagging KNN
0	Number of Drug-Related Crimes	4.93636e+07	5.126989e+07
1	Number of Economically Motivated Crimes	2.276928e+08	2.337125e+08
2	Number of Miscellaneous Crimes	1.802980e+06	2.606275e+06
3	Number of Property-Related Crimes	8.181538e+06	1.154192e+07
4	Number of Public Crime Crimes	2.990289e+05	1.010942e+06
5	Number of Violent Crimes	2.774874e+07	2.617607e+07
6	Number of Vulnerable Populations Crimes	1.527823e+05	1.616028e+05

- Test MSE for Number of Drug-Related Crimes (Bagging): 51269892.82790248
- Test MSE for Number of Economically Motivated Crimes (Bagging): 233712470.80579707
- Test MSE for Number of Miscellaneous Crimes (Bagging): 2606274.6196996197
- Test MSE for Number of Property-Related Crimes (Bagging): 11541921.9374055
- Test MSE for Number of Public Crime Crimes (Bagging): 1010941.5455044294
- Test MSE for Number of Violent Crimes (Bagging): 26176073.18503278

```
In [40]: Y_pred = model.predict(features_test)

# MSE's for the Linear model for each cost of Living target
for i, col in enumerate(targets_test.columns):
    mse = mean_squared_error(targets_test.iloc[:, i], Y_pred)
    print(f'Mean Squared Error for {col}: {mse}')
```

Mean Squared Error for Number of Drug-Related Crimes: 1005668502.8876117
Mean Squared Error for Number of Economically Motivated Crimes: 1739101280.1997714
Mean Squared Error for Number of Miscellaneous Crimes: 10835455.556699539
Mean Squared Error for Number of Property-Related Crimes: 53791482.48202681
Mean Squared Error for Number of Public Crime Crimes: 47126905.35387164
Mean Squared Error for Number of Violent Crimes: 244431371.5096954
Mean Squared Error for Number of Vulnerable Populations Crimes: 99718.30974972651

```
In [41]: regression_mse = [
1005668502.8876117,
1739101280.1997714,
10835455.556699539,
53791482.48202681,
47126905.35387164,
244431371.5096954,
99718.30974972651
]

mse_df['regression all MSE'] = regression_mse

mse_df
```

Out[41]:

	crime category	KNN Model	bootstrapping and bagging KNN	regression all MSE
0	Number of Drug-Related Crimes	4.936368e+07	5.126989e+07	1.005669e+09
1	Number of Economically Motivated Crimes	2.276928e+08	2.337125e+08	1.739101e+09
2	Number of Miscellaneous Crimes	1.802980e+06	2.606275e+06	1.083546e+07
3	Number of Property-Related Crimes	8.181538e+06	1.154192e+07	5.379148e+07
4	Number of Public Crime Crimes	2.990289e+05	1.010942e+06	4.712691e+07
5	Number of Violent Crimes	2.774874e+07	2.617607e+07	2.444314e+08
6	Number of Vulnerable Populations Crimes	1.527823e+05	1.616028e+05	9.971831e+04

- Mean Squared Error for Number of Drug-Related Crimes: 1005668502.8876117
- Mean Squared Error for Number of Economically Motivated Crimes: 1739101280.1997714
- Mean Squared Error for Number of Miscellaneous Crimes: 10835455.556699539
- Mean Squared Error for Number of Property-Related Crimes: 53791482.48202681
- Mean Squared Error for Number of Public Crime Crimes: 47126905.35387164
- Mean Squared Error for Number of Violent Crimes: 244431371.5096954
- Mean Squared Error for Number of Vulnerable Populations Crimes: 99718.30974972651

-KNN and Regression MSE comparisons:

- Number of Drug-Related Crimes:
- KNN Model: 49,363,680.5
- Regression: 1,005,668,502.89 (Higher MSE in Regression)

- Number of Economically Motivated Crimes:
- KNN Model: 227,692,820.11
- Regression: 1,739,101,280.20 (Higher MSE in Regression)

- Number of Miscellaneous Crimes:

Average Monthly
Net Salary (After Tax) \
Number of Drug-Related Crimes
-11.548682
Number of Economically Motivated Crimes
15.766311
Number of Miscellaneous Crimes
1.922924
Number of Property-Related Crimes
1.600657
Number of Public Crime Crimes
-1.102282
Number of Violent Crimes
7.653257
Number of Vulnerable Populations Crimes
0.215472

Mortgage Interest Rate in Percentages (%), Yearly, for 20 Years Fixed-Rate \
Number of Drug-Related Crimes
21747.588493
Number of Economically Motivated Crimes
-2942.119509
Number of Miscellaneous Crimes
1165.267829
Number of Property-Related Crimes
913.081952
Number of Public Crime Crimes
3600.858685
Number of Violent Crimes
6974.534404
Number of Vulnerable Populations Crimes
182.578815

Average Rent Price \
Number of Drug-Related Crimes
026
Number of Economically Motivated Crimes
256
Number of Miscellaneous Crimes
735
Number of Property-Related Crimes
035
Number of Public Crime Crimes
449
Number of Violent Crimes
413
Number of Vulnerable Populations Crimes
855

Average per Square Meter Buy Price \
Number of Drug-Related Crimes
-11.856756
Number of Economically Motivated Crimes
-24.093902
Number of Miscellaneous Crimes
-0.756500
Number of Property-Related Crimes
-5.504281
Number of Public Crime Crimes
0.015463

- KNN Model: 1,802,980.0
- Regression: 10,835,455.56 (Higher MSE in Regression)

- Number of Property-Related Crimes:
- KNN Model: 8,181,537.61
- Regression: 53,791,482.48 (Higher MSE in Regression)

- Number of Public Crime Crimes:
- KNN Model: 299,028.94
- Regression: 47,126,905.35 (Higher MSE in Regression)

- Number of Violent Crimes:
- KNN Model: 27,748,742.94
- Regression: 244,431,371.51 (Higher MSE in Regression)

- Number of Vulnerable Populations Crimes:
- KNN Model: 152,782.28
- Regression: 99,718.31 (Higher MSE in Regression)

In [42]: coefficients = pd.DataFrame(model.coef_, columns=feature
print(coefficients)

Number of Violent Crimes
-9.290196
Number of Vulnerable Populations Crimes
-0.050681

Total Average Cost of all Utilities \
Number of Drug-Related Crimes
24.482236
Number of Economically Motivated Crimes
-2.005885
Number of Miscellaneous Crimes
-39.405916
Number of Property-Related Crimes
29.501786
Number of Public Crime Crimes
25.423513
Number of Violent Crimes
-21.636653
Number of Vulnerable Populations Crimes
-7.140569

Average Restaurant Meal Price \
Number of Drug-Related Crimes
4726.596488
Number of Economically Motivated Crimes
-937.983206
Number of Miscellaneous Crimes
-865.199660
Number of Property-Related Crimes
155.796041
Number of Public Crime Crimes
565.266596
Number of Violent Crimes
-458.456196
Number of Vulnerable Populations Crimes
-92.744102

Average Drink Price \
Number of Drug-Related Crimes
7753
Number of Economically Motivated Crimes
5331
Number of Miscellaneous Crimes
4285
Number of Property-Related Crimes
2190
Number of Public Crime Crimes
7267
Number of Violent Crimes
5811
Number of Vulnerable Populations Crimes
1206

Total Average Cost of all Groceries \
Number of Drug-Related Crimes
-654.163569
Number of Economically Motivated Crimes
-691.576847
Number of Miscellaneous Crimes
125.030150

Number of Property-Related Crimes
-56.818367
Number of Public Crime Crimes
-62.139671
Number of Violent Crimes
-67.988904
Number of Vulnerable Populations Crimes
8.280196

ivity Price
Number of Drug-Related Crimes
2381.075577
Number of Economically Motivated Crimes
1220.533534
Number of Miscellaneous Crimes
326.117713
Number of Property-Related Crimes
-536.266874
Number of Public Crime Crimes
-283.843153
Number of Violent Crimes
-310.324209
Number of Vulnerable Populations Crimes
44.915468

- Average Monthly Net Salary (After Tax):
 - Regression Coefficient: -11.5487
 - Correlation: -0.8099 Both the regression coefficient and the correlation suggest a negative relationship. An increase in net salary is associated with a decrease in drug-related crimes.
- Mortgage Interest Rate in Percentages (%), Yearly, for 20 Years Fixed-Rate:
 - Regression Coefficient: 21747.5885
 - Correlation: 0.5181 The regression coefficient suggests a positive relationship, whereas the correlation suggests a weaker positive relationship. An increase in mortgage interest rates is associated with an increase in drug-related crimes.
- Average Rent Price:
 - Regression Coefficient: 57.4880
 - Correlation: -0.8817 Both the regression coefficient and the correlation suggest a negative relationship. Higher rent prices are associated with a decrease in drug-related crimes.
- Total Average Cost of all Utilities:
 - Regression Coefficient: -11.8568
 - Correlation: -0.6220 Both the regression coefficient and the correlation suggest a negative relationship. Higher property buy prices are associated with a decrease in drug-related crimes.
- Average Restaurant Meal Price:
 - Regression Coefficient: 4726.5965
 - Correlation: -0.9030 Both the regression coefficient and the correlation suggest a strong negative relationship. Higher restaurant meal prices are associated with a significant decrease in drug-related crimes.
- Average Drink Price:
 - Regression Coefficient: -42916.9778
 - Correlation: -0.8729 Both the regression coefficient and the correlation suggest a strong negative relationship. Higher drink prices are associated with a significant decrease in drug-related crimes.
- Total Average Cost of all Groceries:
 - Regression Coefficient: -654.1636
 - Correlation: -0.7370 Both the regression coefficient and the correlation suggest a negative relationship. Higher grocery costs are associated with a decrease in drug-related crimes.
- Average Fun Activity Price:
 - Regression Coefficient: -2381.0756
 - Correlation: -0.1843 Both the regression coefficient and the correlation suggest a negative relationship, but the correlation is weaker. Higher fun activity prices are associated with a decrease in drug-related crimes.

```
In [43]: features2 = total_crime[['Average Monthly Net Salary (After Tax)', 'Mortgage Interest Rate in Percentages (%), Yearly, for 20 Years Fixed-Rate', 'Average Rent Price', 'Average per Square Meter Buy Price', 'Total Average Cost of all Utilities', 'Average Restaurant Meal Price', 'Average Drink Price', 'Total Average Cost of all Groceries', 'Average Fun Activity Price']]

targets2 = total_crime[['Total Crime Numbers']]

train_set2 = total_crime.loc[2010:2020]

test_set2 = total_crime.loc[2021:2022]

# Features for training and testing sets
features_train2 = train_set2[features2.columns]
features_test2 = test_set2[features2.columns]

# Targets for training and testing sets
targets_train2 = train_set2[targets2.columns]
targets_test2 = test_set2[targets2.columns]

model12 = LinearRegression()
model12.fit(features_train2, targets_train2)

Y_pred = model12.predict(features_test2)

# MSE's for the linear model for each cost of living type
for i, col in enumerate(targets2.columns):
    mse = mean_squared_error(targets_test2.iloc[:, i], Y_pred)
    print(f"Mean Squared Error for {col}: {mse}")
```

Mean Squared Error for Total Crime Numbers: 4407006704.844739

With such a large MSE, it makes logical sense to focus more on specific crime types rather than the total number of crimes.

```
In [44]: regression_total_mse = 4407006704.844739

mse_total_crime_df['regression all MSE'] = [regression_total_mse]

mse_total_crime_df
```

Out[44]:

KNN Model	regression all MSE
0.7807916485	4.407007e+09

- Average per Square Meter Buy Price:
 - Regression Coefficient: -11.8568
 - Correlation: -0.6220 Both the regression coefficient and the correlation suggest a negative relationship. Higher property buy prices are associated with a decrease in drug-related crimes.
- Total Average Cost of all Utilities:
 - Regression Coefficient: 24.4822
 - Correlation: 0.1623 Both the regression coefficient and the correlation suggest a positive relationship, but the correlation is weaker. An increase in utility costs is associated with an increase in drug-related crimes.
- Average Restaurant Meal Price:
 - Regression Coefficient: 4726.5965
 - Correlation: -0.9030 Both the regression coefficient and the correlation suggest a strong negative relationship. Higher restaurant meal prices are associated with a significant decrease in drug-related crimes.
- Average Drink Price:
 - Regression Coefficient: -42916.9778
 - Correlation: -0.8729 Both the regression coefficient and the correlation suggest a strong negative relationship. Higher drink prices are associated with a significant decrease in drug-related crimes.
- Total Average Cost of all Groceries:
 - Regression Coefficient: -654.1636
 - Correlation: -0.7370 Both the regression coefficient and the correlation suggest a negative relationship. Higher grocery costs are associated with a decrease in drug-related crimes.
- Average Fun Activity Price:
 - Regression Coefficient: -2381.0756
 - Correlation: -0.1843 Both the regression coefficient and the correlation suggest a negative relationship, but the correlation is weaker. Higher fun activity prices are associated with a decrease in drug-related crimes.

Mean Squared Error for Total Crime Numbers:
4407006704.844739

The MSE is a large number, indicating that the model's predictions for total crime numbers have a substantial amount of error.

In [45]:

	Average Monthly Net Salary (After Tax) \ Total Crime Numbers
	14.507656
	Mortgage Interest Rate in Percentages (%), Yearly, for 20 Years Fixed-Rate \ Total Crime Numbers
	31641.790669
	Average Rent Price \ Total Crime Numbers
	57.4880
	Average per Square Meter Buy Price \ Total Crime Numbers
	-51.536853
	Total Average Cost of all Utilities \ Total Crime Numbers
	512.9218
	Average Restaurant Meal Price \ Total Crime Numbers
	-61702.382859
	Total Average Cost of all Groceries \ Total Crime Numbers
	012.-1398.569
	Average Fun Activity Price \ Total Crime Numbers
	-4361.010166

- Average Monthly Net Salary (After Tax): 14.507656
 - A one-unit increase in average monthly net salary (after tax) is associated with an increase of approximately 14.51 in the total number of crimes.
- Mortgage Interest Rate: 31641.790669
 - A one-unit increase in the yearly mortgage interest rate for a 20-year fixed-rate mortgage is associated with a substantial increase of

approximately 31,641.79 in the total number of crimes.

- Average Rent Price: 75.356229
 - A one-unit increase in average rent price is associated with a modest increase of approximately 75.36 in the total number of crimes.
- Average per Square Meter Buy Price: -51.536853
 - A one-unit increase in the average per square meter buy price is associated with a decrease of approximately 51.54 in the total number of crimes.
- Total Average Cost of all Utilities: 9.218512
 - A one-unit increase in the total average cost of all utilities is associated with a small increase of approximately 9.22 in the total number of crimes.
- Average Restaurant Meal Price: 3093.27596
 - A one-unit increase in the average restaurant meal price is associated with a substantial increase of approximately 3,093.28 in the total number of crimes.
- Average Drink Price: -61702.382859
 - A one-unit increase in the average drink price is associated with a significant decrease of approximately 61,702.38 in the total number of crimes.
- Total Average Cost of all Groceries: -1398.569012
 - A one-unit increase in the total average cost of all groceries is associated with a decrease of approximately 1,398.57 in the total number of crimes.
- Average Fun Activity Price: -4361.010166
 - A one-unit increase in the average fun activity price is associated with a decrease of approximately 4,361.01 in the total number of crimes.

```
In [49]: best_rf = RandomForestRegressor(**best_params, random_state=42)
best_rf.fit(X_train, y_train)

y_pred = best_rf.predict(X_test)
mse = mean_squared_error(y_test, y_pred, multioutput='raw_values')

for i, col in enumerate(targets.columns):
    print(f"Mean Squared Error for {col}: {mse[i]}")
```

```
Mean Squared Error for Number of Drug-Related Crimes: 79515463.92578985
Mean Squared Error for Number of Economically Motivated Crimes: 383381316.88982487
Mean Squared Error for Number of Miscellaneous Crimes: 6632522.45724448
Mean Squared Error for Number of Property-Related Crimes: 33162656.604361083
Mean Squared Error for Number of Public Crime Crimes: 1955679.6870472215
Mean Squared Error for Number of Violent Crimes: 57497107.931304656
Mean Squared Error for Number of Vulnerable Populations Crimes: 89247.96845462959
```

```
In [50]: random_forest_mse = [
79515463.92578985,
383381316.88982487,
6632522.45724448,
33162656.604361083,
1955679.6870472215,
57497107.931304656,
89247.96845462959
]

mse_df['random forest MSE'] = random_forest_mse

mse_df
```

2.3.3 Random Forests and Gradient Boosting

- Objective: Predict crime rates based on cost of living and crime data. Techniques: Random Forests and Gradient Boosting. - Implementation: Both Random Forests and Gradient Boosting are versatile techniques that can be used for both regression and classification tasks.

Random Forests

```
In [46]: from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split

features = col_and_cr_per_year[['Average Monthly Net
                                Mortgage Interest R
                                Average Rent Price'
                                Average per Square
                                Total Average Cost
                                Average Restaurant
                                Average Drink Price
                                Total Average Cost
                                Average Fun Activit

target = col_and_cr_per_year[['Number of Drug-Related
                                Number of Economical
                                Number of Miscellaneous
                                Number of Property-Related
                                Number of Public Crime
                                Number of Violent Crime
                                Number of Vulnerable
```

```
In [47]: from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import GridSearchCV

param_grid = {
    'n_estimators': [10, 50, 100, 200], # Number of
    'max_depth': [None, 10, 20, 30], # Maximum depth
    'min_samples_split': [2, 5, 10], # Minimum number of
    'min_samples_leaf': [1, 2, 4] # Minimum number of
}

rf = RandomForestRegressor(random_state=42)

grid_search = GridSearchCV(estimator=rf, param_grid=param_grid, cv=5)
```

```
In [48]: X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                            test_size=0.2,
                                                            random_state=42)

grid_search.fit(X_train, y_train)

best_params = grid_search.best_params_
print("Best parameters:", best_params)

Fitting 3 folds for each of 144 candidates, totalling 432 fits
Best parameters: {'max_depth': None, 'min_samples_leaf': 2, 'min_samples_split': 2, 'n_estimators': 10}
```

	crime category	KNN Model	bootstrapping and bagging KNN	regression all MSE
0	Number of Drug-Related Crimes	4.936368e+07	5.126989e+07	1.005669e+09
1	Number of Economically Motivated Crimes	2.276928e+08	2.337125e+08	1.739101e+09
2	Number of Miscellaneous Crimes	1.802980e+06	2.606275e+06	1.083546e+07
3	Number of Property-Related Crimes	8.181538e+06	1.154192e+07	5.379148e+07
4	Number of Public Crime Crimes	2.990289e+05	1.010942e+06	4.712691e+07
5	Number of Violent Crimes	2.774874e+07	2.617607e+07	2.444314e+08
6	Number of Vulnerable Populations Crimes	1.527823e+05	1.616028e+05	9.971831e+04

The MSE for "Economically Motivated Crimes" and "Drug-Related Crimes" remains relatively high, though there is a noticeable change. This suggests some improvement, but these types of crimes still present challenges for accurate prediction. The MSE for "Public Crime Crimes" and "Vulnerable Populations Crimes" are relatively low, indicating better model accuracy in predicting these crime counts. The decrease in MSE for "Violent Crimes" and "Property-Related Crimes" after tuning indicates an improvement in the model's ability to predict these crime types, though it is not great.

Feature Importances

```
In [51]: feature_importances = best_rf.feature_importances_
importances_df = pd.DataFrame(feature_importances,
                               index=targets.index,
                               columns=targets.columns)
print(importances_df.sort_values(by="Importance", ascending=False))
```

Importance
Average Rent Price
0.309059
Average Restaurant Meal Price
0.299232
Total Average Cost of all Groceries
0.285993
Mortgage Interest Rate in Percentages (%), Year...
0.091241
Average per Square Meter Buy Price
0.008759
Total Average Cost of all Utilities
0.005717
Average Monthly Net Salary (After Tax)
0.000000
Average Drink Price
0.000000
Average Fun Activity Price
0.000000

Top Influential Features:

Average Rent Price, Average Restaurant Meal Price, and Total Average Cost of all Groceries are the most influential features. They have the highest importance scores, indicating a strong association with the crime rates across the categories modeled. This suggests that these aspects of the cost of living may have a significant impact on crime rates.

Moderately Influential Feature:

Mortgage Interest Rate in Percentages (%) holds moderate importance. This suggests that it has some influence on crime rates, but not as strong as the top features.

Less Influential Features:

Average per Square Meter Buy Price and Total Average Cost of all Utilities have lower importance scores, indicating that they are less influential in the model's predictions of crime rates compared to the top features.

No Influence Features:

Surprisingly, Average Monthly Net Salary (After Tax), Average Drink Price, and Average Fun Activity Price have zero importance in this model. This outcome suggests that, according to the model's logic, these factors do not significantly contribute to predicting crime rates in the dataset. This goes against the correlations derived earlier.

Implementing Features of Importance

```
In [52]: important_features = col_and_cr_per_year[['Average  
'Average  
'Total A  
targets = col_and_cr_per_year[['Number of Drug-Reli  
'Number of Economici  
'Number of Miscellai  
'Number of Property-  
'Number of Public Ci  
'Number of Violent (i  
'Number of Vulnerab  
X_train, X_test, y_train, y_test = train_test_spl  
model = RandomForestRegressor(random_state=42)  
model.fit(X_train, y_train)  
y_pred = model.predict(X_test)  
mse = mean_squared_error(y_test, y_pred, multioutp
```

```
# Print MSE for each target  
for i, col in enumerate(targets.columns):  
    print(f"Mean Squared Error for {col}: {mse[i]}")  
  
Mean Squared Error for Number of Drug-Related Crim  
es: 51738663.03656665  
Mean Squared Error for Number of Economically Moti  
vated Crimes: 33222368.0872333  
Mean Squared Error for Number of Miscellaneous Cri  
mes: 6055555.5019666655  
Mean Squared Error for Number of Property-Related  
Crimes: 27014220.93529998  
Mean Squared Error for Number of Public Crime Crim  
es: 1231890.604133328  
Mean Squared Error for Number of Violent Crimes: 5  
0024305.95820001  
Mean Squared Error for Number of Vulnerable Popula  
tions Crimes: 72624.4016000001
```

```
In [53]: FOIRandom_forest_mse = [  
51738663.03656665,  
33222368.0872333,  
6055555.5019666655,  
27014220.93529998,  
1231890.604133328,  
50024305.95820001,  
72624.4016000001,  
]  
  
mse_df['features of importance random forest MSE']  
  
mse_df
```

Out[53]:

	crime category	KNN Model	bootstrapping and bagging KNN	regression all MSE
0	Number of Drug-Related Crimes	4.936368e+07	5.126989e+07	1.005669e+09
1	Number of Economically Motivated Crimes	2.276928e+08	2.337125e+08	1.739101e+09
2	Number of Miscellaneous Crimes	1.802980e+06	2.606275e+06	1.083546e+07
3	Number of Property- Related Crimes	8.181538e+06	1.154192e+07	5.379148e+07
4	Number of Public Crime Crimes	2.990289e+05	1.010942e+06	4.712691e+07
5	Number of Violent Crimes	2.774874e+07	2.617607e+07	2.444314e+08
6	Number of Vulnerable Populations Crimes	1.527823e+05	1.616028e+05	9.971831e+04

Focusing on Economically Motivated Crimes to
attempt better MSEs:

```
In [54]: from sklearn.model_selection import train_test_spl  
from sklearn.ensemble import RandomForestRegressor  
from sklearn.metrics import mean_squared_error  
  
features = col_and_cr_per_year[['Average Monthly Ne  
'Mortgage Interest  
'Average Rent Price  
'Average per Squar  
'Total Average Cost  
'Average Restaurant  
'Average Drink Pri  
'Total Average Cost  
'Average Fun Activ  
target_econ = col_and_cr_per_year[['Number of Econ  
X_train, X_test, y_train, y_test = train_test_spl  
  
# fit the RandomForestRegressor model  
model = RandomForestRegressor(random_state=42)  
model.fit(X_train, y_train.values.ravel()) # Using
```

```

y_pred = model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)

print("Mean Squared Error:", mse)

Mean Squared Error: 171987539.51236653

In [55]: param_grid = {
    'n_estimators': [100, 200, 300],
    'max_depth': [10, 20, 30, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

from sklearn.model_selection import GridSearchCV

grid_search = GridSearchCV(estimator=RandomForestRegressor(),
                           param_grid=param_grid,
                           cv=3, # Number of cross-validation folds
                           n_jobs=-1,
                           verbose=2,
                           scoring='neg_mean_squared_error')

grid_search.fit(X_train, y_train.values.ravel())

best_params = grid_search.best_params_
print("Best parameters:", best_params)

# Train with the best parameters
best_rf = RandomForestRegressor(**best_params, random_state=42)
best_rf.fit(X_train, y_train.values.ravel())

y_pred = best_rf.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
print("Optimized Mean Squared Error:", mse)

Fitting 3 folds for each of 108 candidates, totalling 324 fits
Best parameters: {'max_depth': 10, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 100}
Optimized Mean Squared Error: 171987539.51236653

```

```

In [56]: #using the correlations from above to redo the model
selected_features = col_and_cr_per_year[['Average Monthly Mortgage Interest',
    'Average Rent Price',
    'Average per Square Foot Total Average Cost',
    'Average Restaurant',
    'Average Drink Price',
    'Total Average Cost',
    'Average Fun Activities']]

# Split the data
X_train, X_test, y_train, y_test = train_test_split(selected_features, col_and_cr_per_year,
                                                    test_size=0.2,
                                                    random_state=42)

# Train the model
model = RandomForestRegressor(random_state=42)
model.fit(X_train, y_train.values.ravel())

# Predict and evaluate
y_pred = model.predict(X_test)
optimized_mse = mean_squared_error(y_test, y_pred)
print("Optimized Mean Squared Error:", optimized_mse)

```

```

# Calculate Mean Squared Error
mse_ridge = mean_squared_error(y_test, y_pred_ridge)
print("Ridge Regression MSE:", mse_ridge)

Ridge Regression MSE: 124275134.12024717

```

Regardless of technique, the MSE for economically motivated crimes remains extremely high.

Gradient Boosting

```

In [60]: features = col_and_cr_per_year[['Average Monthly Mortgage Interest',
    'Average Rent Price',
    'Average per Square Foot Total Average Cost',
    'Average Restaurant',
    'Average Drink Price',
    'Total Average Cost',
    'Average Fun Activities']]

target = col_and_cr_per_year[['Number of Drug-Related Crimes',
    'Number of Economically Motivated Crimes',
    'Number of Miscellaneous Crimes',
    'Number of Property-Related Crimes',
    'Number of Public Crime Crimes',
    'Number of Violent Crimes',
    'Number of Vulnerable Populations Crimes']]

In [61]: from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(features, target,
                                                    test_size=0.2,
                                                    random_state=42)

from sklearn.ensemble import GradientBoostingRegressor
from sklearn.metrics import mean_squared_error

mse_results = {}
for column in target.columns:

    gbr = GradientBoostingRegressor(random_state=42)
    gbr.fit(X_train, y_train[column])

    y_pred = gbr.predict(X_test)

    mse_results[column] = mean_squared_error(y_test[column], y_pred)

for crime_type, mse in mse_results.items():
    print(f"Mean Squared Error for {crime_type}: {mse}")

```

Optimized Mean Squared Error: 223730854.2498667

```

In [57]: #redoing the refinement
new_param_grid = {
    'n_estimators': [100, 150, 200],
    'max_depth': [5, 10, 15, 20],
    'min_samples_split': [2, 3, 4],
    'min_samples_leaf': [1, 2, 3]
}

new_grid_search = GridSearchCV(estimator=RandomForestRegressor(),
                               param_grid=new_param_grid,
                               cv=3, # Number of cross-validation folds
                               n_jobs=-1,
                               verbose=2,
                               scoring='neg_mean_squared_error')

new_grid_search.fit(X_train, y_train.values.ravel())

new_best_params = new_grid_search.best_params_
print("New best parameters:", new_best_params)

new_best_rf = RandomForestRegressor(**new_best_params, random_state=42)
new_best_rf.fit(X_train, y_train.values.ravel())

new_y_pred = new_best_rf.predict(X_test)
new_optimized_mse = mean_squared_error(y_test, new_y_pred)
print("New Optimized Mean Squared Error:", new_optimized_mse)

Fitting 3 folds for each of 108 candidates, totalling 324 fits
New best parameters: {'max_depth': 5, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 100}
New Optimized Mean Squared Error: 223730854.2498667

```

```

In [58]: #Linear Regression

from sklearn.linear_model import LinearRegression

# Initialize and train the Linear Regression model
lin_reg = LinearRegression()
lin_reg.fit(X_train, y_train)

# Predict on the test set
y_pred_lin = lin_reg.predict(X_test)

# Calculate Mean Squared Error
mse_lin = mean_squared_error(y_test, y_pred_lin)
print("Linear Regression MSE:", mse_lin)

Linear Regression MSE: 201522900.42000362

```

```

In [59]: #Ridge Regression

from sklearn.linear_model import Ridge

# Initialize and train the Ridge Regression model
# alpha is the regularization strength; larger values result in more regularization
ridge_reg = Ridge(alpha=1.0)
ridge_reg.fit(X_train, y_train)

# Predict on the test set
y_pred_ridge = ridge_reg.predict(X_test)

```

Mean Squared Error for Number of Drug-Related Crimes: 29184443.356781524
Mean Squared Error for Number of Economically Motivated Crimes: 416199829.3355019
Mean Squared Error for Number of Miscellaneous Crimes: 2446515.056312264
Mean Squared Error for Number of Property-Related Crimes: 11247262.274608037
Mean Squared Error for Number of Public Crime Crimes: 425313.47602445935
Mean Squared Error for Number of Violent Crimes: 23276021.398681138
Mean Squared Error for Number of Vulnerable Populations Crimes: 37680.9259613533

```

In [62]: train_set = col_and_cr_per_year.loc[col_and_cr_per_year['crime_type'] != 'Economically Motivated Crimes']
test_set = col_and_cr_per_year.loc[col_and_cr_per_year['crime_type'] == 'Economically Motivated Crimes']

X_train = train_set[features.columns]
y_train = train_set[target.columns]
X_test = test_set[features.columns]
y_test = test_set[target.columns]

from sklearn.ensemble import GradientBoostingRegressor
from sklearn.metrics import mean_squared_error

gbr = GradientBoostingRegressor(random_state=42)

mse_results_new = {}
for column in target.columns:
    gbr.fit(X_train, y_train[column])

    y_pred = gbr.predict(X_test)

    mse_results_new[column] = mean_squared_error(y_test[column], y_pred)

for crime_type, mse in mse_results_new.items():
    print(f"Mean Squared Error for {crime_type} (new split): {mse}")

```

Mean Squared Error for Number of Drug-Related Crimes (new split): 83982538.58135214
Mean Squared Error for Number of Economically Motivated Crimes (new split): 265574084.30078137
Mean Squared Error for Number of Miscellaneous Crimes (new split): 186326.5586957412
Mean Squared Error for Number of Property-Related Crimes (new split): 8181936.312585218
Mean Squared Error for Number of Public Crime Crimes (new split): 65402.00879314674
Mean Squared Error for Number of Violent Crimes (new split): 4817044.991733102
Mean Squared Error for Number of Vulnerable Populations Crimes (new split): 31204.49084871357

```

In [63]: gradient_boost_mse = [
    83982538.58135214,
    265574084.30078137,
    186326.5586957412,
    8181936.312585218,
    65402.00879314674,

```



```
4817044.991733102,
31204.49084871357,
]

mse_df['gradient boosting MSE'] = gradient_boost_mse
mse_df
```

Out[63]:

	crime category	KNN Model	bootstrapping and bagging KNN	regression all MSE
0	Number of Drug-Related Crimes	4.936368e+07	5.126989e+07	1.005669e+09
1	Number of Economically Motivated Crimes	2.276928e+08	2.337125e+08	1.739101e+09
2	Number of Miscellaneous Crimes	1.802980e+06	2.606275e+06	1.083546e+07
3	Number of Property-Related Crimes	8.181538e+06	1.154192e+07	5.379148e+07
4	Number of Public Crime Crimes	2.990289e+05	1.010942e+06	4.712691e+07
5	Number of Violent Crimes	2.774874e+07	2.617607e+07	2.444314e+08
6	Number of Vulnerable Populations Crimes	1.527823e+05	1.616028e+05	9.971831e+04

In [64]: `#mse_df.to_csv('mse.csv', index=False)`

2.3.4 Forward Selection Linear Regression on Individual Crime Categories

- Objective: Reduce the dimensionality of the data and identify key components for individual crime categories. - Implementation: Forward selection linear regression is used to build a model by iteratively adding the most significant predictor variables. This

```
new_pval = pd.Series(index=remaining_features,
                      data=[float('inf')] * len(remaining_features))

for new_column in remaining_features:
    model = sm.OLS(y, sm.add_constant(X[new_column]))
    new_pval[new_column] = model.pvalues[new_column]

min_p_value = new_pval.min()
if min_p_value < significance_level:
    best_features.append(new_pval.idxmin())
else:
    break

return best_features
```

Economically Motivated Crimes

```
In [69]: X = col_and_cr_per_year[['Average Monthly Net Salary',
                                  'Mortgage Interest Rate in Percentages (%)',
                                  'Average Rent Price',
                                  'Average per Square Meter Buy Price',
                                  'Total Average Cost of all Utilities',
                                  'Average Restaurant Meal Price',
                                  'Average Drink Price',
                                  'Total Average Cost of all Groceries',
                                  'Average Fun Activity Price',
                                  'Number of Drug-Related Crimes',
                                  'Number of Economically Motivated Crimes',
                                  'Number of Miscellaneous Crimes',
                                  'Number of Property-Related Crimes',
                                  'Number of Public Crime Crimes',
                                  'Number of Violent Crimes',
                                  'Number of Vulnerable Populations Crimes']]

y = col_and_cr_per_year['Number of Economically Motivated Crimes']

selected_features = forward_selection(X, y)
print("Selected features:", selected_features)

model = sm.OLS(y, sm.add_constant(X[selected_features]))
print(model.summary())
```

approach helps reduce the number of variables and identifies the key components driving crime rates for each individual crime category.

```
In [65]: print(col_and_cr_per_year.columns.tolist())

['Average Monthly Net Salary (After Tax)', 'Mortgage Interest Rate in Percentages (%)', 'Yearly, for 20 Years Fixed-Rate', 'Average Rent Price', 'Average per Square Meter Buy Price', 'Total Average Cost of all Utilities', 'Average Restaurant Meal Price', 'Average Drink Price', 'Total Average Cost of all Groceries', 'Average Fun Activity Price', 'Number of Drug-Related Crimes', 'Number of Economically Motivated Crimes', 'Number of Miscellaneous Crimes', 'Number of Property-Related Crimes', 'Number of Public Crime Crimes', 'Number of Violent Crimes', 'Number of Vulnerable Populations Crimes']
```

```
In [66]: import statsmodels.api as sm

In [67]: import pandas as pd

crime_columns = [
    'Number of Drug-Related Crimes',
    'Number of Economically Motivated Crimes',
    'Number of Miscellaneous Crimes',
    'Number of Property-Related Crimes',
    'Number of Public Crime Crimes',
    'Number of Violent Crimes',
    'Number of Vulnerable Populations Crimes'
]

forward_selection = pd.DataFrame({'Crime Type': crime_columns})

forward_selection
```

Out[67]:

	Crime Type
0	Number of Drug-Related Crimes
1	Number of Economically Motivated Crimes
2	Number of Miscellaneous Crimes
3	Number of Property-Related Crimes
4	Number of Public Crime Crimes
5	Number of Violent Crimes
6	Number of Vulnerable Populations Crimes

Linear Regression Per Crime Category

```
In [68]: def forward_selection(X, y, significance_level=0.05):
    initial_features = X.columns.tolist()
    best_features = []

    while len(initial_features) > 0:
        remaining_features = list(set(initial_features) - set(best_features))
```

Selected features: ['Average Restaurant Meal Price', 'Average per Square Meter Buy Price', 'Mortgage Interest Rate in Percentages (%)', 'Yearly, for 20 Years Fixed-Rate']

OLS Regression Results

```
=====
Dep. Variable:      Number of Economically Motivated Crimes
R-squared:           0.942
Model:
OLS
Adj. R-squared:      0.923
Method:
Least Squares
F-statistic:         48.89
Date:                Thu, 14 Dec 2023
Prob (F-statistic):  6.78e-06
Time:
19:51:39
Log-Likelihood:      -129.13
No. Observations:    266.3
Df Residuals:        13
Df Model:            268.5
Covariance Type:     nonrobust
=====
```

coef	std err	t	P> t	[0.025	0.975]	

const						
1.702e+05	1.91e+04	8.904	0.000	1.27e+05	2.13e+05	
Average Restaurant Meal Price						
-2066.0347	415.645	-4.971	0.001	-3006.289	-1125.781	
Average per Square Meter Buy Price						
-10.5676	2.998	-3.525	0.006	-17.349	-3.786	
Mortgage Interest Rate in Percentages (%), Yearly, for 20 Years Fixed-Rate	8085.1381	2710.477	2.983	0.015	1953.614	1.42e+04
=====						
Omnibus:			0.761	Durbin-Watson:		
Prob(Omnibus):	1.467		0.684	Jarque-Bera (JB):		
Skew:	0.701		0.323	Prob(JB):		
Kurtosis:	0.704		2.064	Cond. No.		
3.30e+04						
=====						

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 3.3e+04. This may indicate that the model is ill conditioned and standard errors of the parameter estimates may be large and unreliable.

ight indicate that there are strong multicollinearity or other numerical problems.

```
C:\Users\brina\anaconda3\Lib\site-packages\scipy\stats\_stats_py.py:1806: UserWarning: kurtosistest only valid for n>=20 ... continuing anyway, n=13
warnings.warn("kurtosistest only valid for n>=20 ... continuing ")
```

```
In [70]: selected_features = [
None,
['Average Restaurant Meal Price', 'Average per
None,
None,
None,
None,
None]

r2_values = [
None,
0.923,
None,
None,
None,
None,
None]

forward_selection = pd.DataFrame({'Crime Type': cr
forward_selection
```

	Crime Type	Selected Features	R2
0	Number of Drug-Related Crimes	None	NaN
1	Number of Economically Motivated Crimes	[Average Restaurant Meal Price, Average per Sq...	0.923
2	Number of Miscellaneous Crimes	None	NaN
3	Number of Property-Related Crimes	None	NaN
4	Number of Public Crime Crimes	None	NaN
5	Number of Violent Crimes	None	NaN
6	Number of Vulnerable Populations Crimes	None	NaN

Selected Features:

The forward selection process identified these features as significant predictors: Average Restaurant Meal Price

	VIF	feat
ure		
0	Average Monthly Net Salary (After Tax)	403.331698
1	Mortgage Interest Rate in Percentages (%), Yearly, for 20 Years Fixed-Rate	146.764640
2	Average Rent Per Square Meter Buy Price	438.393853
3	Average per Square Meter Buy Price	61.858113
4	Total Average Cost of all Utilities	236.658727
5	Average Restaurant Meal Price	813.505805
6	Average Drink Price	1104.062306
7	Total Average Cost of all Groceries	554.289038
8	Average Fun Activity Price	180.376069

The Variance Inflation Factor (VIF) values for the features are significantly high, indicating strong multicollinearity among them.

```
In [72]: X_modified = X.drop('Average Drink Price', axis=1)

vif_data_modified = pd.DataFrame()
vif_data_modified['feature'] = X_modified.columns
vif_data_modified['VIF'] = [variance_inflation_fa

print(vif_data_modified)
```

	VIF	feat
ure		
0	Average Monthly Net Salary (After Tax)	400.938545
1	Mortgage Interest Rate in Percentages (%), Yearly, for 20 Years Fixed-Rate	136.722278
2	Average Rent Per Square Meter Buy Price	417.424796
3	Average per Square Meter Buy Price	57.906297
4	Total Average Cost of all Utilities	234.457142
5	Average Restaurant Meal Price	537.089165
6	Total Average Cost of all Groceries	421.971700
7	Average Fun Activity Price	180.142982

```
In [73]: X_modified_2 = X_modified.drop('Average Restaurant Meal Price', axis=1)

vif_data_modified_2 = pd.DataFrame()
vif_data_modified_2['feature'] = X_modified_2.col
vif_data_modified_2['VIF'] = [variance_inflation_

print(vif_data_modified_2)
```

Average per Square Meter Buy Price Mortgage Interest Rate in Percentages (%), Yearly, for 20 Years Fixed-Rate

Coefficient Interpretation:

Average Restaurant Meal Price: The negative coefficient (-2066.0347) suggests that as the price of restaurant meals increases, the number of economically motivated crimes decreases. This could indicate an inverse relationship between dining costs and economically motivated crimes.

Average per Square Meter Buy Price: Similarly, the negative coefficient (-10.5676) indicates that higher property prices are associated with fewer economically motivated crimes.

Mortgage Interest Rate: The positive coefficient (8085.1381) suggests that higher mortgage interest rates are associated with an increase in economically motivated crimes.

```
In [71]: # checking for multicollinearity

from statsmodels.stats.outliers_influence import

# Calculate VIF for each feature
vif_data = pd.DataFrame()
vif_data['feature'] = X.columns
vif_data['VIF'] = [variance_inflation_factor(X.va

print(vif_data)
```

	VIF	feat
ure		
0	Average Monthly Net Salary (After Tax)	143.190612
1	Mortgage Interest Rate in Percentages (%), Yearly, for 20 Years Fixed-Rate	136.716704
2	Average Rent Per Square Meter Buy Price	416.976851
3	Average per Square Meter Buy Price	51.944838
4	Total Average Cost of all Utilities	181.904106
5	Total Average Cost of all Groceries	354.233312
6	Average Fun Activity Price	117.464609

```
In [74]: from sklearn.linear_model import Ridge

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_modified_2)

ridge_model = Ridge(alpha=1.0)
ridge_model.fit(X_scaled, y)
```

```
Out[74]: * Ridge
Ridge()
```

Principal Component Analysis (PCA) for additional information

```
In [75]: from sklearn.decomposition import PCA

pca = PCA().fit(X_scaled)

import matplotlib.pyplot as plt

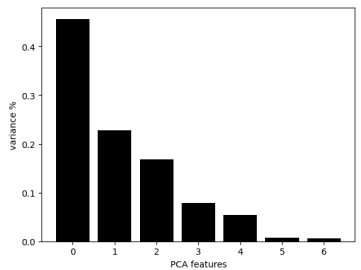
features = range(pca.n_components_)
plt.bar(features, pca.explained_variance_ratio_,
plt.xlabel('PCA features')
plt.ylabel('variance %')
plt.xticks(features)
plt.show()

cumulative_variance = np.cumsum(pca.explained_var

# Determine the number of components to explain 9
number_of_components = np.where(cumulative_varian
print("Number of components to explain 95% varian

pca_optimal = PCA(n_components=number_of_componen
X_pca = pca_optimal.fit_transform(X_scaled)

model_pca = sm.OLS(y, sm.add_constant(X_pca)).fit
print(model_pca.summary())
```



Number of components to explain 95% variance: 5
OLS Regression Results

=====

Dep. Variable: Number of Economically Motivated Crimes
R-squared: 0.895

Model: OLS
Adj. R-squared: 0.820
Method: Least Squares
F-statistic: 11.95

Date: Thu, 14 Dec 2023
Prob (F-statistic): 0.00255

Time: 19:51:39
Log-Likelihood: -133.00

No. Observations: 13
AIC: 278.0
Df Residuals: 7
BIC: 281.4
Df Model: 5

Covariance Type: nonrobust

=====

	coef	std err	t
P> t	[0.025	0.975]	
const	1.195e+05	2536.861	47.106
0.000	1.14e+05	1.25e+05	
x1	1.051e+04	1418.873	7.404
0.000	7149.997	1.39e+04	
x2	-870.3161	2007.246	-0.434
0.678	-5616.698	3876.066	
x3	4966.8383	2336.664	2.126
0.071	-558.494	1.05e+04	
x4	-1626.8850	3414.495	-0.476
0.648	-9700.882	6447.112	
x5	44.8461	4112.959	0.011
0.992	-9680.757	9770.449	

=====

Omnibus: 0.024 Durbin-Watson: 1.595
Prob(Omnibus): 0.988 Jarque-Bera (JB): 0.239
Skew: 0.052 Prob(JB): 0.887
Kurtosis: 2.344 Cond. No. 2.90

=====

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
C:\Users\brina\anaconda3\Lib\site-packages\scipy\stats\_stats.py:1806: UserWarning: kurtosistest only valid for n>=20 ... continuing anyway, n=13
warnings.warn("kurtosistest only valid for n>=20 ... continuing ")

In [76]: from sklearn.decomposition import PCA
from sklearn.linear_model import LinearRegression

X = col_and_cr_per_year[['Average Monthly Net Sal',
'Mortgage Interest Rate',
'Average Rent Price',
'Average per Square Meter',
'Total Average Cost of a Restaurant Meal',
'Average Drink Price',
'Total Average Cost of a Fun Activity Per Person']]

y = col_and_cr_per_year['Number of Economically Motivated Crimes']

desired_variance_explained = 0.95

pca = PCA(n_components=desired_variance_explained)
X_pca = pca.fit_transform(X)

model = LinearRegression()
model.fit(X_pca, y)

feature_importance_scores = pca.components_

# feature importance scores
print("Feature Importance Scores:")
print(feature_importance_scores)

Feature Importance Scores:
[[-7.59126121e-01  3.24601670e-04 -2.50893052e-01
-6.00572578e-01
  4.93001675e-03 -4.41011524e-03 -3.29769842e-04
-6.98681098e-03
-6.36504767e-05]
 [ 6.14827825e-01  1.09041050e-04  2.43205608e-02
-7.86974811e-01
  4.53262964e-02  2.57991256e-03  2.79299223e-04
 1.96612421e-03
  9.04271181e-04]]

In [77]: X2 = col_and_cr_per_year[['Average Monthly Net Sa',
'Mortgage Interest Rate',
'Average Rent Price',
'Average per Square Meter',
'Total Average Cost of a Restaurant Meal',
'Average Drink Price',
'Total Average Cost of a Fun Activity Per Person']]

pca = PCA().fit(X2)

cumulative_variance = np.cumsum(pca.explained_variance_ratio_)

number_of_components = np.where(cumulative_variance >= 0.95)[0][0] + 1
```

```
print("Number of components to explain 95% variance: ", number_of_components)

pca_optimal = PCA(n_components=number_of_components)
X_pca = pca_optimal.fit_transform(X2)

model_pca = sm.OLS(y, sm.add_constant(X_pca)).fit()
print(model_pca.summary())
```

```
Number of components to explain 95% variance: 2
OLS Regress

sion Results
=====
=
Dep. Variable:      Number of Economically Motivat
ed Crimes      R-squared:                0.80
0
Model:
OLS      Adj. R-squared:                0.760
Method:      Least
t Squares      F-statistic:                20.0
5
Date:                Thu, 14
Dec 2023      Prob (F-statistic):        0.000317
Time:
19:51:39      Log-Likelihood:            -137.18
No. Observations:
13      AIC:                280.4
Df Residuals:
10      BIC:                282.1
Df Model:
2
Covariance Type:
nonrobust
=====
=====
coef      std err      t
P>|t|      [0.025      0.975]
-----
const      1.195e+05      2928.338      40.808
0.000      1.13e+05      1.26e+05
x1          17.8062      2.833      6.285
0.000      11.494      24.119
x2          4.7305      6.134      0.771
0.458      -8.937      18.398
=====
Omnibus:                1.978      Durbin-Wa
tson:                1.767
Prob(Omnibus):          0.372      Jarque-B
e ra (JB):          0.476
Skew:                -0.422      Prob(JB):
0.788
Kurtosis:                3.407      Cond. No.
1.03e+03
=====
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 1.03e+03. This might indicate that there are strong multicollinearity or other numerical problems.

```
Selected features: ['Average Restaurant Meal Price']
OLS Regression

Results
=====
Dep. Variable:      Number of Drug-Related Crimes
R-squared:                0.815
Model:      OLS
Adj. R-squared:        0.799
Method:      Least Squares
F-statistic:          48.59
Date:                Thu, 14 Dec 2023
Prob (F-statistic):    2.36e-05
Time:                19:51:39
Log-Likelihood:        -130.61
No. Observations:      13
AIC:                265.2
Df Residuals:          11
BIC:                266.3
Df Model:              1
Covariance Type:      nonrobust
=====
=====
coef      std e
rr      t      P>|t|      [0.025      0.975]
-----
const      8.364e+04      9110.1
75      9.181      0.000      6.36e+04      1.04e+05
Average Restaurant Meal Price -2245.6606      322.1
49      -6.971      0.000      -2954.705      -1536.616
=====
Omnibus:                1.016      Durbin-Wa
tson:                1.219
Prob(Omnibus):          0.602      Jarque-B
e ra (JB):          0.718
Skew:                -0.523      Prob(JB):
0.698
Kurtosis:                2.518      Cond. No.
153.
=====
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

C:\Users\brina\anaconda3\Lib\site-packages\scipy\stats_stats.py:1806: UserWarning: kurtosistest only valid for n>=20 ... continuing anyway, n=13

warnings.warn("kurtosistest only valid for n>=20 ... continuing ")

```
In [80]: selected_features2 = [
        ['Average Restaurant Meal Price'],
        ['Average Restaurant Meal Price', 'Average pe
        None,
        None,
        None,
        None,
        None
    ]
```

C:\Users\brina\anaconda3\Lib\site-packages\scipy\stats_stats.py:1806: UserWarning: kurtosistest only valid for n>=20 ... continuing anyway, n=13

warnings.warn("kurtosistest only valid for n>=20 ... continuing ")

Drug- Related Crimes

```
In [78]: #redefining for continuation

def forward_selection(X, y, significance_level=0.05):
    initial_features = X.columns.tolist()
    best_features = []

    while len(initial_features) > 0:
        remaining_features = list(set(initial_features))
        new_pval = pd.Series(index=remaining_features, dtype=float)

        for new_column in remaining_features:
            model = sm.OLS(y, sm.add_constant(X[remaining_features + [new_column]]))
            new_pval[new_column] = model.pvalues[new_column]

        min_p_value = new_pval.min()
        if min_p_value < significance_level:
            best_features.append(new_pval.idxmin())
        else:
            break

    return best_features
```

```
In [79]: X = col_and_cr_per_year[['Average Monthly Net Salary',
        'Mortgage Interest Rate in Percentages (%)',
        'Average Rent Price',
        'Average per Square Meter Buy Price',
        'Total Average Cost of all Utilities',
        'Average Restaurant Meal Price',
        'Average Drink Price',
        'Total Average Cost of all Groceries',
        'Average Fun Activity Price']

y = col_and_cr_per_year['Number of Drug-Related Crimes']

selected_features = forward_selection(X, y)
print("Selected features:", selected_features)

model = sm.OLS(y, sm.add_constant(X[selected_features]))
print(model.summary())
```

```
r2_values2 = [
    0.799,
    0.923,
    None,
    None,
    None,
    None,
    None,
    None
]

forward_selection = pd.DataFrame({'Crime Type': crime_type, 'R2': r2_values2})
forward_selection
```

	Crime Type	Selected Features	R2
0	Number of Drug-Related Crimes	[Average Restaurant Meal Price]	0.799
1	Number of Economically Motivated Crimes	[Average Restaurant Meal Price, Average per Square Meter Buy Price]	0.923
2	Number of Miscellaneous Crimes	None	NaN
3	Number of Property-Related Crimes	None	NaN
4	Number of Public Crime Crimes	None	NaN
5	Number of Violent Crimes	None	NaN
6	Number of Vulnerable Populations Crimes	None	NaN

```
In [81]: vif_data = pd.DataFrame()
vif_data['feature'] = X.columns
vif_data['VIF'] = [variance_inflation_factor(X.values, i) for i in range(X.columns)]
print(vif_data)

feature      VIF
0      Average Monthly Net Salary (After Tax)      403.331698
1      Mortgage Interest Rate in Percentages (%), Yearly
a...      146.764640
2      Average Rent Price
ice      438.393853
3      Average per Square Meter Buy Price
ice      61.858113
4      Total Average Cost of all Utilities
ies      236.658727
5      Average Restaurant Meal Price
ice      813.505805
6      Average Drink Price
ice      1104.062306
7      Total Average Cost of all Groceries
ies      554.289038
8      Average Fun Activity Price
ice      180.376069
```

Miscellaneous Crimes

```
In [82]: #redefining for continuation

def forward_selection(X, y, significance_level=0.05):
    initial_features = X.columns.tolist()
    best_features = []

    while len(initial_features) > 0:
        remaining_features = list(set(initial_features))
        new_pval = pd.Series(index=remaining_features, dtype=float)

        for new_column in remaining_features:
            model = sm.OLS(y, sm.add_constant(X[best_features + [new_column]]))
            new_pval[new_column] = model.pvalues[new_column]

        min_p_value = new_pval.min()
        if min_p_value < significance_level:
            best_features.append(new_pval.idxmin())
        else:
            break

    return best_features

In [83]: X = col_and_cr_per_year[['Average Monthly Net Salary',
    'Mortgage Interest Rate',
    'Average Rent Price',
    'Average per Square Meter Price',
    'Total Average Cost of a Meal',
    'Average Restaurant Meal Price',
    'Average Drink Price',
    'Total Average Cost of a Meal',
    'Average Fun Activity Price']

y = col_and_cr_per_year['Number of Miscellaneous Crimes']

selected_features = forward_selection(X, y)
print("Selected features:", selected_features)

model = sm.OLS(y, sm.add_constant(X[selected_features]))
print(model.summary())
```

Selected features: ['Average Restaurant Meal Price', 'Average per Square Meter Buy Price']

OLS Regression Results

=====				
Dep. Variable:	Number of Miscellaneous Crimes			
R-squared:	0.886			
Model:	OLS			
Adj. R-squared:	0.863			
Method:	Least Squares			
F-statistic:	38.71			
Date:	Thu, 14 Dec 2023			
Prob (F-statistic):	1.96e-05			
Time:	19:51:39			
Log-Likelihood:	-105.28			
No. Observations:	13			
AIC:	216.6			
Df Residuals:	10			
BIC:	218.3			
Df Model:	2			
Covariance Type:	nonrobust			
=====				
====				
	std err	t	P> t	coef
				[0.025 0.975]

const				2.88e+04 1
371.847	20.995	0.000	2.57e+04	3.19e+04
Average Restaurant Meal Price				-311.0933
61.121	-5.090	0.000	-447.280	-174.907
Average per Square Meter Buy Price				-1.1091
0.440	-2.521	0.030	-2.090	-0.129
=====				
Omnibus:	1.066	Durbin-Watson:	1.941	
Prob(Omnibus):	0.587	Jarque-Bera (JB):	0.895	
Skew:	-0.523	Prob(JB):	0.639	
Kurtosis:	2.252	Cond. No.	1.55e+04	
=====				
Notes:				
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.				
[2] The condition number is large, 1.55e+04. This might indicate that there are strong multicollinearity or other numerical problems.				

C:\Users\brina\anaconda3\Lib\site-packages\scipy\stats_stats_py.py:1806: UserWarning: kurtosistest only valid for n>=20 ... continuing anyway, n=13

warnings.warn("kurtosistest only valid for n>=20 ... continuing ")

```
In [84]: selected_features3 = [
    ['Average Restaurant Meal Price'],
    ['Average Restaurant Meal Price', 'Average per Square Meter Price'],
    ['Average Restaurant Meal Price', 'Average per Square Meter Price', 'Average Drink Price'],
    None,
    None,
    None,
    None
]

r2_values3 = [
    0.799,
    0.923,
    0.863,
    None,
    None,
    None,
    None
]

forward_selection = pd.DataFrame({'Crime Type': crime_type, 'Selected Features': selected_features3, 'R2': r2_values3})

forward_selection
```

Out[84]:

	Crime Type	Selected Features	R2
0	Number of Drug-Related Crimes	[Average Restaurant Meal Price]	0.799
1	Number of Economically Motivated Crimes	[Average Restaurant Meal Price, Average per Square Meter Price]	0.923
2	Number of Miscellaneous Crimes	[Average Restaurant Meal Price, Average per Square Meter Price, Average Drink Price]	0.863
3	Number of Property-Related Crimes	None	NaN
4	Number of Public Crime Crimes	None	NaN
5	Number of Violent Crimes	None	NaN
6	Number of Vulnerable Populations Crimes	None	NaN

Property-Related Crimes

```
In [85]: #redefining for continuation

def forward_selection(X, y, significance_level=0.05):
    initial_features = X.columns.tolist()
    best_features = []
```

```
while len(initial_features) > 0:
    remaining_features = list(set(initial_features))
    new_pval = pd.Series(index=remaining_features, dtype=float)

    for new_column in remaining_features:
        model = sm.OLS(y, sm.add_constant(X[best_features + [new_column]]))
        new_pval[new_column] = model.pvalues[new_column]

    min_p_value = new_pval.min()
    if min_p_value < significance_level:
        best_features.append(new_pval.idxmin())
    else:
        break

    return best_features

In [86]: X = col_and_cr_per_year[['Average Monthly Net Salary',
    'Mortgage Interest Rate',
    'Average Rent Price',
    'Average per Square Meter Price',
    'Total Average Cost of a Meal',
    'Average Restaurant Meal Price',
    'Average Drink Price',
    'Total Average Cost of a Meal',
    'Average Fun Activity Price']

y = col_and_cr_per_year['Number of Property-Related Crimes']

selected_features = forward_selection(X, y)
print("Selected features:", selected_features)

model = sm.OLS(y, sm.add_constant(X[selected_features]))
print(model.summary())
```

Selected features: ['Average Restauraunt Meal Price', 'Average per Square Meter Buy Price', 'Mortgage Interest Rate in Percentages (%), Yearly, for 20 Years Fixed-Rate']

```
OLS Regression Results
=====
Dep. Variable:      Number of Property-Related Crimes
R-squared:          0.968
Model:
OLS
Adj. R-squared:     0.958
Method:             Least Squares
F-statistic:        91.70
Date:               Thu, 14 Dec 2023
Prob (F-statistic): 4.58e-07
Time:              19:5
1:39 Log-Likelihood: -109.48
No. Observations:   13
AIC:                227.0
Df Residuals:       9
BIC:                229.2
Df Model:           3
Covariance Type:    nonrobust
=====
```

	coef	std err	t	P> t	[0.025	0.975]
const	5.795e+04	4218.500	13.736	0.000	4.84e+04	6.75e+04
Average Restauraunt Meal Price	-690.8002	91.733	-7.531	0.000	-898.314	-483.286
Average per Square Meter Buy Price	-3.2978	0.662	-4.984	0.001	-4.795	-1.801
Mortgage Interest Rate in Percentages (%), Yearly, for 20 Years Fixed-Rate	2.736	0.023	283.739	2990.192		
Omnibus:			2.617	Durbin-Watson:		
Prob(Omnibus):		2.384		0.270	Jarque-Bera (JB):	
Skew:		1.009		0.022	Prob(JB):	
0.604						
Kurtosis:			1.636	Cond. No.		
3.30e+04						

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 3.3e+04. This

might indicate that there are strong multicollinearity or other numerical problems.

```
C:\Users\brina\anaconda3\Lib\site-packages\scipy\stats\_stats.py:1806: UserWarning: kurtosistest only valid for n>=20 ... continuing anyway, n=13
warnings.warn("kurtosistest only valid for n>=20 ... continuing ")
```

```
In [87]: selected_features4 = [
        ['Average Restauraunt Meal Price'],
        ['Average Restauraunt Meal Price'], 'Average per Square Meter Buy Price',
        ['Average Restauraunt Meal Price'], 'Average per Square Meter Buy Price',
        ['Average Restauraunt Meal Price'], 'Average per Square Meter Buy Price',
        None,
        None,
        None,
        None
    ]

r2_values4 = [
    0.799,
    0.923,
    0.863,
    0.958,
    None,
    None,
    None,
    None
]

forward_selection = pd.DataFrame({'Crime Type': crime_type, 'R2': r2_values4})

forward_selection
```

	Crime Type	Selected Features	R2
0	Number of Drug-Related Crimes	[Average Restauraunt Meal Price]	0.799
1	Number of Economically Motivated Crimes	[Average Restauraunt Meal Price, Average per Square Meter Buy Price]	0.923
2	Number of Miscellaneous Crimes	[Average Restauraunt Meal Price, Average per Square Meter Buy Price, Mortgage Interest Rate in Percentages (%), Yearly, for 20 Years Fixed-Rate]	0.863
3	Number of Property-Related Crimes	[Average Restauraunt Meal Price, Average per Square Meter Buy Price, Mortgage Interest Rate in Percentages (%), Yearly, for 20 Years Fixed-Rate]	0.958
4	Number of Public Crime Crimes	None	NaN
5	Number of Violent Crimes	None	NaN
6	Number of Vulnerable Populations Crimes	None	NaN

Public Crime

```
In [88]: #redefining for continuation
```

```
def forward_selection(X, y, significance_level=0.05):
    initial_features = X.columns.tolist()
    best_features = []

    while len(initial_features) > 0:
        remaining_features = list(set(initial_features) - set(best_features))
        new_pval = pd.Series(index=remaining_features, dtype=float)

        for new_column in remaining_features:
            model = sm.OLS(y, sm.add_constant(X[best_features + [new_column]]))
            new_pval[new_column] = model.pvalues[new_column]

        min_p_value = new_pval.min()
        if min_p_value < significance_level:
            best_features.append(new_pval.idxmin())
        else:
            break

    return best_features
```

```
In [89]: X = col_and_cr_per_year[['Average Monthly Net Salary (After Tax)',
    'Mortgage Interest Rate',
    'Average Rent Price',
    'Average per Square Meter Buy Price',
    'Total Average Cost of a Home',
    'Average Restauraunt Meal Price',
    'Average Drink Price',
    'Total Average Cost of a Home',
    'Average Fun Activity Price']

y = col_and_cr_per_year['Number of Public Crime Crimes']

selected_features = forward_selection(X, y)
print("Selected features:", selected_features)

model = sm.OLS(y, sm.add_constant(X[selected_features]))
print(model.summary())
```

```
C:\Users\brina\anaconda3\Lib\site-packages\scipy\stats\_stats.py:1806: UserWarning: kurtosistest only valid for n>=20 ... continuing anyway, n=13
warnings.warn("kurtosistest only valid for n>=20 ... continuing ")
```

Selected features: ['Average Monthly Net Salary (After Tax)']

```
OLS Regression Results
=====
Dep. Variable:      Number of Public Crime Crimes
R-squared:          0.456
Model:
OLS
Adj. R-squared:     0.407
Method:             Least Squares
F-statistic:        9.227
Date:               Thu, 14 Dec 2023
Prob (F-statistic): 0.0113
Time:              19:51:39
Log-Likelihood:     -107.09
No. Observations:   13
AIC:                218.2
Df Residuals:       11
BIC:                219.3
Df Model:           1
Covariance Type:    nonrobust
=====
```

	coef	std err	t	P> t	[0.025	0.975]
const	4985.1720	1267.172	3.934	0.002	2196.146	7774.198
Average Monthly Net Salary (After Tax)	0.9995	0.329	3.038	0.011	0.275	1.724
Omnibus:			3.198	Durbin-Watson:		
Prob(Omnibus):		1.011		0.202	Jarque-Bera (JB):	
Skew:		1.311		-0.760	Prob(JB):	
0.519						
Kurtosis:			3.332	Cond. No.		
1.77e+04						

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 1.77e+04. This might indicate that there are strong multicollinearity or other numerical problems.

```
In [90]: selected_features5 = [
        ['Average Restauraunt Meal Price'],
        ['Average Restauraunt Meal Price'], 'Average per Square Meter Buy Price',
        ['Average Restauraunt Meal Price'], 'Average per Square Meter Buy Price',
        ['Average Restauraunt Meal Price'], 'Average per Square Meter Buy Price',
        ['Average Monthly Net Salary (After Tax)'],
        None,
        None,
        None,
        None
    ]
```

```
None,
None
]

r2_values5 = [
    0.799,
    0.923,
    0.863,
    0.958,
    0.407,
    None,
    None
]

forward_selection = pd.DataFrame({'Crime Type': c
forward_selection
```

Out[90]:

	Crime Type	Selected Features	R2
0	Number of Drug-Related Crimes	[Average Restaunt Meal Price]	0.799
1	Economically Motivated Crimes	[Average Restaunt Meal Price, Average per Sq...	0.923
2	Number of Miscellaneous Crimes	[Average Restaunt Meal Price, Average per Sq...	0.863
3	Number of Property-Related Crimes	[Average Restaunt Meal Price, Average per Sq...	0.958
4	Number of Public Crime Crimes	[Average Monthly Net Salary (After Tax)]	0.407
5	Number of Violent Crimes	None	NaN
6	Number of Vulnerable Populations Crimes	None	NaN

Violent Crimes

```
In [91]: #redefining for continuation

def forward_selection(X, y, significance_level=0.
    initial_features = X.columns.tolist()
    best_features = []

    while len(initial_features) > 0:
        remaining_features = list(set(initial_fea
        new_pval = pd.Series(index=remaining_feat

        for new_column in remaining_features:
            model = sm.OLS(y, sm.add_constant(X[b
            new_pval[new_column] = model.pvalues[

        min_p_value = new_pval.min()
        if min_p_value < significance_level:
            best_features.append(new_pval.idxmin(
        else:
            break
```

Selected features: ['Average Restaunt Meal Price', 'Average per Square Meter Buy Price']

OLS Regression Results

=====

Dep. Variable: Number of Violent Crimes R-squared: 0.885 OLS Adj R-squared: 0.861 OLS

Method: Least Squares F-statistic: 38.29 Prob (F-statistic): 2.06e-05

Date: Thu, 14 Dec 2023 Time: 19:51:39 Log Likelihood: -120.76

No. Observations: 13 AI C: 247.5 BI C: 249.2

Df Model: 2 Df Residuals: 10 Covariance Type: nonrobust

=====

coef

std err t P>|t| [0.025 0.975]

const 1.112e+05 4.512 24.634 0.000 1.01e+05 1.21e+05

Average Restaunt Meal Price -847.5004 201.045 -4.215 0.002 -1295.456 -399.544

Average per Square Meter Buy Price -4.9882 1.447 -3.447 0.006 -8.213 -1.763

=====

Omnibus: 1.472 Durbin-Watson: 1.296 Prob(Omnibus): 0.479 Jarque-Bera (JB): 0.797 Skew: 0.033 Prob(JB): 0.671 Kurtosis: 1.789 Cond. No. 1.55e+04

=====

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 1.55e+04. This might indicate that there are strong multicollinearity or other numerical problems.

```
return best_features

In [92]: X = col_and_cr_per_year[['Average Monthly Net Sal
    'Mortgage Interest Rate',
    'Average Rent Price',
    'Average per Square Meter',
    'Total Average Cost of a Meal',
    'Average Restaunt Meal Price',
    'Average Drink Price',
    'Total Average Cost of a Meal',
    'Average Fun Activity Price']

y = col_and_cr_per_year['Number of Violent Crimes']

selected_features = forward_selection(X, y)
print("Selected features:", selected_features)

model = sm.OLS(y, sm.add_constant(X[selected_features]))
print(model.summary())
```

Out[92]:

	Crime Type	Selected Features	R2
0	Number of Drug-Related Crimes	[Average Restaunt Meal Price]	0.799
1	Economically Motivated Crimes	[Average Restaunt Meal Price, Average per Sq...	0.923
2	Number of Miscellaneous Crimes	[Average Restaunt Meal Price, Average per Sq...	0.863
3	Number of Property-Related Crimes	[Average Restaunt Meal Price, Average per Sq...	0.958
4	Number of Public Crime Crimes	[Average Monthly Net Salary (After Tax)]	0.407
5	Number of Violent Crimes	[Average Restaunt Meal Price, Average per Sq...	0.861
6	Number of Vulnerable Populations Crimes	None	NaN

Vulnerable Populations Crimes

```
In [93]: #redefining for continuation

def forward_selection(X, y, significance_level=0.
    initial_features = X.columns.tolist()
    best_features = []

    while len(initial_features) > 0:
        remaining_features = list(set(initial_fea
        new_pval = pd.Series(index=remaining_feat

        for new_column in remaining_features:
            model = sm.OLS(y, sm.add_constant(X[b
            new_pval[new_column] = model.pvalues[

        min_p_value = new_pval.min()
        if min_p_value < significance_level:
            best_features.append(new_pval.idxmin(
        else:
            break
```

Selected features: ['Average Restaunt Meal Price', 'Average per Square Meter Buy Price']

OLS Regression Results

=====

Dep. Variable: Number of Violent Crimes R-squared: 0.885 OLS Adj R-squared: 0.861 OLS

Method: Least Squares F-statistic: 38.29 Prob (F-statistic): 2.06e-05

Date: Thu, 14 Dec 2023 Time: 19:51:39 Log Likelihood: -120.76

No. Observations: 13 AI C: 247.5 BI C: 249.2

Df Model: 2 Df Residuals: 10 Covariance Type: nonrobust

=====

coef

std err t P>|t| [0.025 0.975]

const 1.112e+05 4.512 24.634 0.000 1.01e+05 1.21e+05

Average Restaunt Meal Price -847.5004 201.045 -4.215 0.002 -1295.456 -399.544

Average per Square Meter Buy Price -4.9882 1.447 -3.447 0.006 -8.213 -1.763

=====

Omnibus: 1.472 Durbin-Watson: 1.296 Prob(Omnibus): 0.479 Jarque-Bera (JB): 0.797 Skew: 0.033 Prob(JB): 0.671 Kurtosis: 1.789 Cond. No. 1.55e+04

=====

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 1.55e+04. This might indicate that there are strong multicollinearity or other numerical problems.

```
In [95]: X = col_and_cr_per_year[['Average Monthly Net Sal
      'Mortgage Interest Rate
      'Average Rent Price',
      'Average per Square Mete
      'Total Average Cost of a
      'Average Restaurent Meal
      'Average Drink Price',
      'Total Average Cost of a
      'Average Fun Activity Pr

y = col_and_cr_per_year['Number of Vulnerable Pop

selected_features = forward_selection(X, y)
print("Selected features:", selected_features)

model = sm.OLS(y, sm.add_constant(X[selected_feat
print(model.summary())
```

Out[98]:	Crime Type	Selected Features	R2
0	Number of Drug-Related Crimes	Restaurant Meal Price	0.799
1	Number of Economically Motivated Crimes	Restaurant Meal PriceHome Buy PriceMortgage In...	0.923
2	Number of Miscellaneous Crimes	Restaurant Meal PriceHome Buy Price	0.863
3	Number of Property-Related Crimes	Restaurant Meal PriceHome Buy PriceMortgage In...	0.958
4	Number of Public Crime Crimes	Monthly Net Salary	0.407
5	Number of Violent Crimes	Restaurant Meal PriceHome Buy Price	0.861
6	Number of Vulnerable Populations Crimes	Monthly Net Salary	0.680

```
In [99]: mse_df = pd.read_csv('mse.csv')

         forward_selection = pd.read_csv('forward_selectio

         forward_selection_short= pd.read_csv('forward_sel
```

This data frame, `mse_df`, can be used to compare the MSEs of all of the models created in the Data Analysis

Out[96]:	Crime Type	Selected Features	R2
0	Number of Drug-Related Crimes	Average Restaurant Meal Price	0.799
1	Number of Economically Motivated Crimes	Average Restaurant Meal PriceAverage per Squar...	0.923
2	Number of Miscellaneous Crimes	Average Restaurant Meal PriceAverage per Squar...	0.863
3	Number of Property-Related Crimes	Average Restaurant Meal PriceAverage per Squar...	0.958
4	Number of Public Crime Crimes	Average Monthly Net Salary (After Tax)	0.407
5	Number of Violent Crimes	Average Restaurant Meal PriceAverage per Squar...	0.861
6	Number of Vulnerable Populations Crimes	Average Monthly Net Salary (After Tax)	0.680

```
In [98]: selected_features8 = [
    'Restaurent Meal Price',
    'Restaurent Meal Price' + 'Home Buy Price' +
    'Restaurent Meal Price' + 'Home Buy Price',
```



```
section.

In [100]: mse_df

Out[100]:
```

	crime category	KNN Model	bootstrapping and bagging KNN	regression all MSE
0	Number of Drug-Related Crimes	4.936368e+07	5.126989e+07	1.005669e+09
1	Number of Economically Motivated Crimes	2.276928e+08	2.337125e+08	1.739101e+09
2	Number of Miscellaneous Crimes	1.802980e+06	2.606275e+06	1.083546e+07
3	Number of Property-Related Crimes	8.181538e+06	1.154192e+07	5.379148e+07
4	Number of Public Crime Crimes	2.990289e+05	1.010942e+06	4.712691e+07
5	Number of Violent Crimes	2.774874e+07	2.617607e+07	2.444314e+08
6	Number of Vulnerable Populations Crimes	1.527823e+05	1.616028e+05	9.971831e+04

```
In [101]: mse_df2 = mse_df.round(0)

mse_df2

mse_df2.set_index('crime category', inplace=True)
mse_df2
```

```
Out[102]:
```

	crime category	KNN Model	bootstrapping and bagging KNN	regression all MSE	n
0	Number of Drug-Related Crimes	49363680	51269893	1005668503	79
1	Number of Economically Motivated Crimes	227692820	233712471	1739101280	383
2	Number of Miscellaneous Crimes	1802980	2606275	10835456	6
3	Number of Property-Related Crimes	8181538	11541922	53791482	33
4	Number of Public Crime Crimes	299029	1010942	47126905	1
5	Number of Violent Crimes	27748743	26176073	244431372	57
6	Number of Vulnerable Populations Crimes	152782	161603	99718	

The above chart for mse_df2, styled_df, visually displays the models that have the lowest MSE values for each crime category. As seen, the KNN model and the Gradient Boosting model have the lowest MSEs for the most crime categories. After evaluating the MSEs, the Gradient Boosting model appears to be the best at predicting the crime rates. However, the MSEs are still extremely large. The training set simply does not have enough data to reliably train the model. With more data, it is possible that the predictive power will increase.

```
In [103]: new_column_names = {
    'KNN Model': 'KNN',
    'bootstrapping and bagging KNN': 'Boot and B',
    'regression all MSE': 'Lin Regression',
}
```

```
Out[101]:
```

	crime category	KNN Model	bootstrapping and bagging KNN	regression all MSE
0	Number of Drug-Related Crimes	49363680.0	51269893.0	1.005669e+09
1	Number of Economically Motivated Crimes	227692820.0	233712471.0	1.739101e+09
2	Number of Miscellaneous Crimes	1802980.0	2606275.0	1.083546e+07
3	Number of Property-Related Crimes	8181538.0	11541922.0	5.379148e+07
4	Number of Public Crime Crimes	299029.0	1010942.0	4.712690e+07
5	Number of Violent Crimes	27748743.0	26176073.0	2.444314e+08
6	Number of Vulnerable Populations Crimes	152782.0	161603.0	9.971800e+04

```
In [102]: def highlight_min(data, color='pink'):
    attr = f'background-color: {color}'
    if data.ndim == 1:
        is_min = data == data.min()
        return [attr if v else '' for v in is_min]
    else:
        is_min = data == data.min().min()
        return pd.DataFrame(np.where(data == is_min,
                                     index=data.index, col=
                                     data.columns, value=attr),
                             index=data.index, columns=
                             data.columns)

styled_df = mse_df2.style.format("{:.0f}")
styled_df = styled_df.apply(highlight_min, axis=1)
styled_df
```

```
Out[103]:
```

	crime category	KNN	Boot and Bag KNN	Lin Regression
0	Number of Drug-Related Crimes	49363680.0	51269893.0	1.005669e+09
1	Number of Economically Motivated Crimes	227692820.0	233712471.0	1.739101e+09
2	Number of Miscellaneous Crimes	1802980.0	2606275.0	1.083546e+07
3	Number of Property-Related Crimes	8181538.0	11541922.0	5.379148e+07
4	Number of Public Crime Crimes	299029.0	1010942.0	4.712690e+07
5	Number of Violent Crimes	27748743.0	26176073.0	2.444314e+08
6	Number of Vulnerable Populations Crimes	152782.0	161603.0	9.971800e+04

The below bar charts visually display the different MSEs for all of the models on all of the crime types. The smaller the MSEs, the more accurate that model is at predicted that crime type rate.

```
In [104]: import matplotlib.pyplot as plt
import numpy as np

plt.figure(figsize=(10, 6))

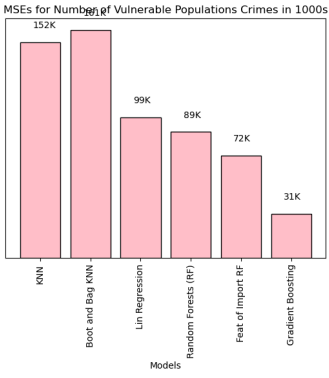
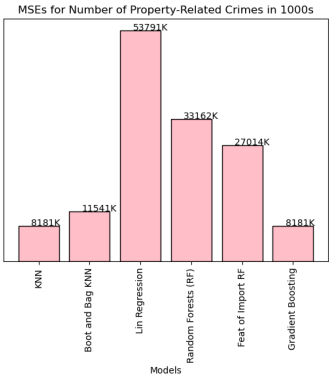
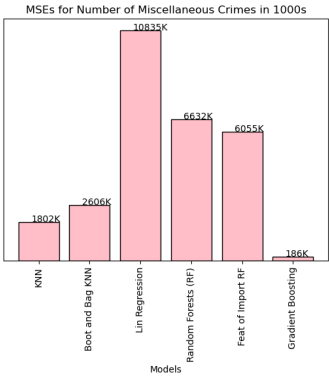
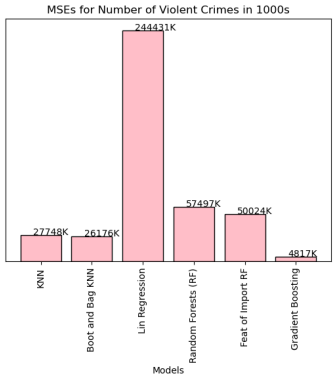
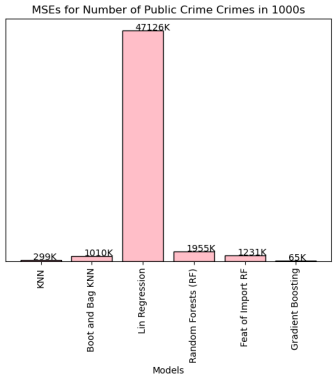
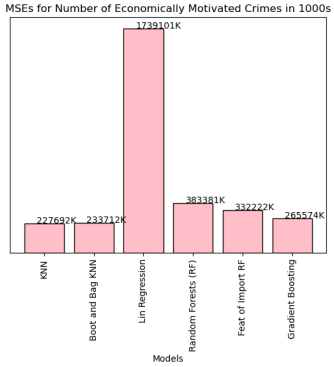
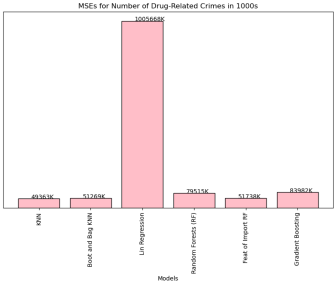
for crime_category, mse_values in mse_df2.iteritems():
    mse_values = mse_values / 1000 # Convert to thousands
    bars = plt.bar(mse_values.index, mse_value
```

```
plt.xlabel('Models')
plt.title(f'MSEs for {crime_category} in 1000s')

for bar, count in zip(bars, mse_values):
    plt.text(bar.get_x() + bar.get_width(), count)

plt.xticks(rotation=90)

plt.show()
```



In [105]:

	Crime Type	Selected Features	R2
0	Number of Drug-Related Crimes	Average Restaurant Meal Price	0.799
1	Number of Economically Motivated Crimes	Average Restaurant Meal PriceAverage per Squar...	0.923
2	Number of Miscellaneous Crimes	Average Restaurant Meal PriceAverage per Squar...	0.863
3	Number of Property-Related Crimes	Average Restaurant Meal PriceAverage per Squar...	0.958
4	Number of Public Crime Crimes	Average Monthly Net Salary (After Tax)	0.407
5	Number of Violent Crimes	Average Restaurant Meal PriceAverage per Squar...	0.861
6	Number of Vulnerable Populations Crimes	Average Monthly Net Salary (After Tax)	0.680

```
crime_types = forward_selection['Crime Type']
selected_features = forward_selection['Selected Features']
r2_values = forward_selection['R2']

fig, ax = plt.subplots(figsize=(12, 6))

bar_width = 0.5

x_indices = np.arange(len(crime_types))
```

```
bar1 = ax.bar(x_indices, r2_values, width=bar_w)

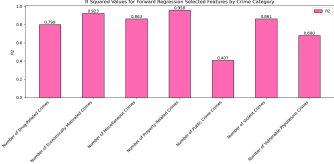
ax.set_xticks(x_indices)
ax.set_xticklabels(crime_types, rotation=45, ha='center')
ax.set_ylabel('R2')

ax.set_title('R Squared Values for Forward Reg')

for i, v in enumerate(r2_values):
    ax.text(i, v, f'{v:.3f}', ha='center', va='bottom')

ax.legend()

plt.tight_layout()
plt.show()
```



This bar chart shows the R-squared values for different categories of crimes based on selected features identified through forward regression analysis. The R-squared value is a statistical measure that represents the proportion of the variance for a dependent variable that's explained by an independent variable or variables in a regression model.

```
In [107]: forward_selection_short
```

Unnamed: 0	Crime Type	Selected Features	R2
0	Number of Drug-Related Crimes	Restaura Meal Price	0.799
1	Number of Economically Motivated Crimes	Restaura Meal PriceHome Buy PriceMortgage In...	0.923
2	Number of Miscellaneous Crimes	Restaura Meal PriceHome Buy Price	0.863
3	Number of Property-Related Crimes	Restaura Meal PriceHome Buy PriceMortgage In...	0.958
4	Number of Public Crime Crimes	Monthly Net Salary	0.407
5	Number of Violent Crimes	Restaura Meal PriceHome Buy Price	0.861
6	Number of Vulnerable Populations Crimes	Monthly Net Salary	0.680

```
In [108]: import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

data = {
    'Crime Type': [
        'Drug-Related Crimes',
        'Economically Motivated Crimes',
        'Economically Motivated Crimes',
        'Economically Motivated Crimes',
        'Economically Motivated Crimes',
        'Miscellaneous Crimes',
        'Miscellaneous Crimes',
        'Property-Related Crimes',
        'Property-Related Crimes',
        'Property-Related Crimes',
        'Public Crime Crimes',
        'Violent Crimes',
        'Violent Crimes',
        'Vulnerable Populations Crimes'
    ],
    'Selected Features': [
        'Restaurant Meal Price',
        'Restaurant Meal Price',
        'Home Buy Price',
        'Mortgage Interest Rate',
        'Mortgage Interest Rate',
        'Mortgage Interest Rate',
        'Mortgage Interest Rate',
        'Mortgage Interest Rate',
        'Mortgage Interest Rate',
        'Mortgage Interest Rate',
        'Mortgage Interest Rate',
        'Mortgage Interest Rate',
        'Mortgage Interest Rate',
        'Mortgage Interest Rate'
    ]
}
```

```
data = {
    'Crime Type': [
        'Drug-Related Crimes',
        'Economically Motivated Crimes',
        'Economically Motivated Crimes',
        'Economically Motivated Crimes',
        'Economically Motivated Crimes',
        'Miscellaneous Crimes',
        'Miscellaneous Crimes',
        'Property-Related Crimes',
        'Property-Related Crimes',
        'Property-Related Crimes',
        'Public Crime Crimes',
        'Violent Crimes',
        'Violent Crimes',
        'Vulnerable Populations Crimes'
    ],
    'Selected Features': [
        'Restaurant Meal Price',
        'Home Buy Price',
        'Restaurant Meal Price',
        'Home Buy Price',
        'Mortgage Interest Rate',
        'Monthly Net Salary',
        'Restaurant Meal Price',
        'Home Buy Price',
        'Monthly Net Salary',
        'Monthly Net Salary',
        'Monthly Net Salary',
        'Monthly Net Salary',
        'Monthly Net Salary',
        'Monthly Net Salary'
    ]
}
```

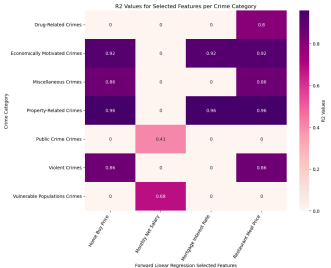
```
df = pd.DataFrame(data)

# Correcting the pivot call
pivot_table = df.pivot(index='Crime Type', columns='Selected Features')

pivot_table = pivot_table.fillna(0)

plt.figure(figsize=(10, 8))
sns.heatmap(pivot_table, annot=True, cmap="RdYlGn")

plt.title('R2 Values for Selected Features per Crime Category')
plt.xlabel('Forward Linear Regression Selected Features')
plt.ylabel('Crime Category')
plt.xticks(rotation=60, ha='right')
plt.show()
```



This heatmap represents the R-squared values of various crime categories based on selected features from a forward linear regression analysis. From this, it is easy to identify which cost of living features were selected for each crime type and their respective R squared values.

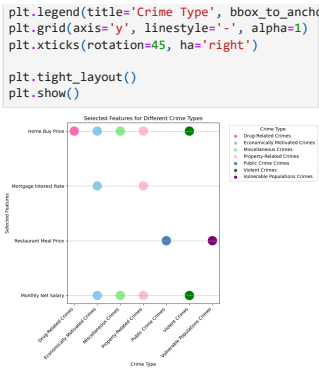
```
In [109]: data = {
    'Crime Type': [
        'Drug-Related Crimes',
        'Economically Motivated Crimes',
        'Economically Motivated Crimes',
        'Economically Motivated Crimes',
        'Economically Motivated Crimes',
        'Miscellaneous Crimes',
        'Miscellaneous Crimes',
        'Property-Related Crimes',
        'Property-Related Crimes',
        'Property-Related Crimes',
        'Public Crime Crimes',
        'Violent Crimes',
        'Violent Crimes',
        'Vulnerable Populations Crimes'
    ],
    'Selected Features': [
        'Restaurant Meal Price',
        'Restaurant Meal Price',
        'Home Buy Price',
        'Mortgage Interest Rate',
        'Restaurant Meal Price',
        'Home Buy Price',
        'Restaurant Meal Price',
        'Home Buy Price',
        'Mortgage Interest Rate',
        'Monthly Net Salary',
        'Restaurant Meal Price',
        'Home Buy Price',
        'Monthly Net Salary',
        'Monthly Net Salary'
    ]
}

forward_selection_short = pd.DataFrame(data)

forward_selection_short['Selected Features'] = forward_selection_short['y_position']

plt.figure(figsize=(10, 8))
scatter = sns.scatterplot(
    x=forward_selection_short['Crime Type'],
    y=forward_selection_short['y_position'],
    hue=forward_selection_short['Crime Type'],
    palette=['hotpink', 'skyblue', 'lightgreen'],
    s=500
)

plt.xticks(ticks=forward_selection_short['Crime Type'])
plt.xlabel('Crime Type')
plt.ylabel('Selected Features')
plt.title('Selected Features for Different Crime Types')
```



Similar to the heatmap, this scatterplot displays which cost of living features were selected for each crime type.

The Gradient Boosting model appears to be the best at predicting the crime rates as it is the model with the lowest MSEs for the most crime types. The MSEs are still extremely large, which is most likely due to the training set simply not having enough data to reliably train the model. As mentioned previously, with more data, it is possible that the predictive power will increase.

As shown in the heatmap and scatterplot, the cost of living features selected for the various crime types include: Home Buy Price, Monthly Net Salary, Mortgage Interest Rates, and Restaurant Meal Price.

- Home Buy Price: For economically and property-related crimes, the home buy price is a significant predictor. This could be because these types of crimes often

The Gradient Boosting model proved to have the lowest MSE values for the most crime categories. Using this model, I am able to build a predictive model in which values can be entered for each of the cost of living features, and then it will output the predicted crime rates of each crime type.

```
In [110]: #Gradient Boosting Model with Lowest MSEs

from sklearn.ensemble import GradientBoos
from sklearn.metrics import mean_squared_

#cost of living features
features = [
    'Average Monthly Net Salary (After Tax)',
    'Mortgage Interest Rate in Percentage',
    'Average Rent Price',
    'Average per Square Meter Buy Price',
    'Total Average Cost of all Utilities',
    'Average Restaurant Meal Price',
    'Average Drink Price',
    'Total Average Cost of all Groceries',
    'Average Fun Activity Price'
]

# crime rate targets
target = [
    'Number of Drug-Related Crimes',
    'Number of Economically Motivated Cri',
    'Number of Miscellaneous Crimes',
    'Number of Property-Related Crimes',
    'Number of Public Crime Crimes',
    'Number of Violent Crimes',
    'Number of Vulnerable Populations Cri'
]

train_set = col_and_cr_per_year.loc[col_and_cr_per_year.index.isin(train_set.index)]
test_set = col_and_cr_per_year.loc[col_and_cr_per_year.index.isin(test_set.index)]

X_train = train_set[features]
y_train = train_set[target]
X_test = test_set[features]
y_test = test_set[target]

gbr_models = {}
mse_results_new = {}

for column in target:
    gbr = GradientBoostingRegressor(random_state=1)
    gbr.fit(X_train, y_train[column])
    y_pred = gbr.predict(X_test)
    mse_results_new[column] = mean_squared_error(y_test, y_pred)
    gbr_models[column] = gbr # Store the model

# MSE
for crime_type, mse in mse_results_new.items():
    print(f'Mean Squared Error for {crime_type}: {mse}')
```

involve financial gain or loss related to property. The value of homes could reflect the economic status of a neighborhood, which in turn may influence the rate of crimes like burglary or theft.

- Monthly Net Salary: This feature is not highlighted as a strong predictor in the heatmap provided, but generally, it could be an important factor. Higher salaries might correlate with lower crime rates as they can lead to better living conditions and reduced economic pressure to engage in criminal activities.
- Mortgage Interest Rate: This feature has shown a strong predictive relationship with economically motivated and property-related crimes. High mortgage rates can strain household finances, potentially leading to more financially driven crimes as individuals struggle with payments.
- Restaurant Meal Price: This is a significant predictor for economically motivated and miscellaneous crimes. The cost of meals at restaurants may serve as a barometer for the overall cost of living and disposable income. When meal prices are high, indicating a higher cost of living, individuals with economic strains might be more inclined to engage in crime.

4. Predictive Model

```
Mean Squared Error for Number of Drug-Related Crimes (new split): 83982538.58135214
Mean Squared Error for Number of Economically Motivated Crimes (new split): 265574084.30078137
Mean Squared Error for Number of Miscellaneous Crimes (new split): 186326.5586957412
Mean Squared Error for Number of Property-Related Crimes (new split): 8181936.312585218
Mean Squared Error for Number of Public Crime Crimes (new split): 65402.00879314674
Mean Squared Error for Number of Violent Crimes (new split): 4817044.991733102
Mean Squared Error for Number of Vulnerable Populations Crimes (new split): 31204.49084871357
```

Below is the predictive model. Any values desired can be entered into the new_feature_values.

- Choose an average monthly net_salary: for example, \$4,000
- Choose an average annual interest rate: for example, 8%
- Choose an average home rental price: for example, \$1950
- Choose an average home buy price per square meter: for example, \$1100
- Choose an utilities price: for example, \$250
- Choose an average restaurant meal price: for example, \$35
- Choose an average drink price: for example, \$5
- Choose an average total grocery price: for example, \$80

- Choose an average fun activity price:
for example, \$25

Enter the chosen values into the predictive model, as shown below. Then, the model will output the predicted crime rates for each crime type.

```
In [111]: new_feature_values = pd.DataFrame([[
    4000, # net_salary
    8, # interest_rate
    1950, # rent_price
    1100, # buy_price
    250, # utilities_cost
    35, # meal_price
    5, # drink_price
    80, # groceries_cost
    25 # fun_activity_price
]], columns=features)

predictions = {}

for crime_type, model in gbr_models.items():
    prediction = model.predict(new_feature_values)
    predictions[crime_type] = prediction

for crime_type, predicted_value in predictions.items():
    print(f"Predicted {crime_type}: {predicted_value}")

Predicted Number of Drug-Related Crimes: 20068.448318382132
Predicted Number of Economically Motivated Crimes: 99656.9447120232
Predicted Number of Miscellaneous Crime s: 16840.693181774463
Predicted Number of Property-Related Crimes: 43637.256308078
Predicted Number of Public Crime Crime s: 9473.353671525236
Predicted Number of Violent Crimes: 78495.50853585123
Predicted Number of Vulnerable Populations Crimes: 2285.0083594613684
```

5. Limitations and Challenges

In the course of this project, several limitations and challenges were encountered that have a significant

potential discrepancies in the data.

- Discrepancies in Government Crime Data Updates:
 - There were significant differences observed in the crime data released by the Chicago government between the updates in April 2023 and December 2023. The most recent update in December appeared to omit substantial amounts of data compared to the previous one. Consequently, the April 2023 dataset was utilized for this project. This decision, while necessary, means that the analysis may not fully capture the most current crime trends or data patterns.
- Limited Data for Training Models:
 - The project faced a constraint in terms of the volume of data available for training the predictive models. Adequate and diverse data is crucial for training robust and accurate machine learning models. The scarcity of data points can lead to models that are not well-generalized and may perform poorly when predicting new or unseen data.
- High Mean Squared Errors (MSEs) in Models:

impact on the results and their interpretation. Understanding these limitations is crucial for accurately assessing the predictive power and applicability of the models developed.

- Reliance on Crowdsourced Data for Cost of Living:
 - The primary dataset for cost of living in Chicago was sourced from Numbeo, which is largely based on crowdsourced information. While crowdsourcing can provide a broad range of data, it inherently carries the risk of inaccuracies and biases. The data may not be a fully representative or accurate reflection of the actual costs in Chicago, as it relies on individual contributions which can vary in reliability and timeliness.
- Use of National Inflation Rate for Missing Values:
 - To address missing values in the cost of living data, the national inflation rate was used as a fill-in. This approach poses a limitation since the national inflation rate may not align precisely with the inflation trends or cost changes specific to Chicago. Different regions can experience varying rates of inflation due to local economic conditions, policies, and other factors, leading to

- Even the best-performing model, the Gradient Boosting Regressor, exhibited high Mean Squared Errors. A high MSE indicates that the model's predictions deviate significantly from the actual values, suggesting limited predictive accuracy. This limitation is critical as it directly impacts the reliability of the forecasts made by the model, casting doubt on its effectiveness in real-world applications.

Each of these limitations underscores the challenges faced in developing predictive models in the context of dynamic and complex urban environments like Chicago. They also highlight the necessity for cautious interpretation of the results and the potential need for further refinement and validation of the models with more comprehensive and accurate data.

6. Conclusions

This research embarked on a comprehensive exploration of the intricate relationship between various cost of living factors and crime rates in the city of Chicago. The primary aim was to unravel how specific cost of living factors uniquely influence diverse categories of crime. Through data analysis and predictive modeling, this research sought to exhumate the underpinnings of crime in relation to

cost of living factors, addressing two core objectives.

Objective 1 focused on examining the correlation between distinct cost of living indicators and the incidence of various crime categories. The analysis delved into a range of cost of living factors, including average monthly net salary, mortgage interest rates, rent, property prices, utility costs, and expenses related to everyday living such as meals, drinks, groceries, and leisure activities. The study revealed significant negative correlations between economic indicators such as average rent price, restaurant meal price, drink price, and grocery costs, and different crime categories, including drug-related, economically motivated, miscellaneous, and property-related crimes. Particularly notable was the strong negative correlation between lower rent and restaurant meal prices and reduced crime rates across several categories. Additionally, higher average monthly net salaries and mortgage interest rates were also found to be inversely correlated with crime rates, suggesting that improved economic conditions could potentially lead to a decrease in crime.

These findings imply that economic factors play a crucial role in influencing crime rates in Chicago. Lower costs in essential living expenses, such as housing and food, appear to be

model and laid the groundwork for further research in this area. The insights gained from this study can serve as a valuable resource for city planners, policymakers, law enforcement, and community organizations. The findings provide a basis for developing targeted crime prevention strategies, efficiently allocating resources, and ultimately fostering a safer and more economically stable urban community. Future research, with access to more comprehensive and accurate data, could build upon this foundation to develop more robust models, furthering our understanding of the complex interplay between cost of living and crime.

7. Future Research

The initial findings and challenges encountered in this study pave the way for future research that can build upon the established groundwork. A crucial aspect of future research should involve the expansion and enrichment of the data set. The current limitations in data quantity and quality have notably constrained the predictive power of the models used. Therefore, incorporating a more comprehensive and detailed data collection would be instrumental in enhancing model accuracy and reliability. Future research should focus on gathering more granular data that captures the

associated with lower crime rates, echoing the importance of economic stability in crime prevention. If these correlations hold true in causal analyses, they could inform critical policy decisions aimed at mitigating crime through economic interventions. However, caution must be exercised in interpreting these correlations, as establishing causality requires more in-depth research.

Objective 2 aimed to develop a predictive model to assess how shifts in cost of living might impact crime rates. Despite the challenges faced, including reliance on crowdsourced data, discrepancies in government crime data, limited training data, and high Mean Squared Errors in predictive models, the project made strides in constructing a Gradient Boosting Regressor model. This model, designed to forecast changes in crime rates in response to cost of living fluctuations, offers a foundation for future refinement and application. Although the predictive power was constrained by data limitations and model performance, this research represents a significant stride towards adopting a data-driven methodology in comprehending and predicting the nuances of crime rate dynamics.

While the project faced several challenges that limited the accuracy and reliability of the predictive models, it successfully developed a predictive

diverse socio-economic facets of life in Chicago, including more nuanced cost of living elements and a broader spectrum of crime statistics.

In addition to expanding the data set, future research should also explore the integration of longitudinal data that tracks economic and crime trends over longer periods. This would provide a deeper understanding of the temporal dynamics of crime in relation to economic fluctuations.

Future studies should also aim to establish causal relationships between economic indicators and crime rates. This could involve the application of econometric models or experimental designs that can better identify causation, rather than mere correlation. Such an approach would offer more actionable insights for policymakers and law enforcement agencies.

Extending the research to include comparative analyses with other cities, both within and outside the United States, could offer a broader perspective on how different urban environments respond to economic changes in the context of crime. This comparative approach would help validate the findings from Chicago and identify unique or common patterns across various urban settings, contributing to a more holistic

understanding of the economic underpinnings of urban crime.

Future research, with more robust and detailed data, has the potential to significantly advance the understanding of the complex interplay between the cost of living and crime. This progression towards a more data-intensive approach in criminology could lead to more effective and targeted crime prevention and intervention strategies, ultimately contributing to safer and more stable urban communities.

8. Theoretical Testing

Inflation Based Extended Data

In order to test to see if more data will improve the predictive power of the model, the inflation rate was used to backfill cost of living data for eight years (2001-2009). This was then combined with crime rates for the same years. The final data set includes cost of living information and crime rates for the different crime types for 2001 - 2022 in Chicago.

```
In [112... #url = 'https://www.usinflationcal
# Retrieved from Part 1
#using inflation information for 2001-2009

df_inflation = [
    ["Year", "Ave"],
    [2023, None],
```

```
columns_to_drop = ['IUCR', 'Beat',
                    'X Coordinate',
                    'Ward', 'Commur',
                    'Description',

crimes_0122 = crime_data_large.drop
```

```
In [114... crime_data_large
```

Out[114]:

	ID	Case Number	Date
	0	10224738	HY411648 2015-09-05 13:30:00
	1	10224739	HY411615 2015-09-04 11:30:00
	2	11646166	JC213529 2018-09-01 00:01:00
	3	10224740	HY411595 2015-09-05 12:45:00
	4	10224741	HY411610 2015-09-05 13:00:00
...	...	...	...
7681519	12847575	JF420478	2022-09-01 05:00:00
7681520	12847801	JF420319	2022-07-08 00:00:00
7681521	12847324	JF420102	2022-09-27 11:00:00
7681522	12847570	JF420427	2022-09-03 10:25:00
7681523	12840464	JF411839	2022-09-26 19:20:00

7681524 rows × 22 columns

```
In [115... crimes_0122.head()
crimes_0122.info()
missing_values = crimes_0122.isna()
print(missing_values)
```

```
[2022, 8.0],
[2021, 4.7],
[2020, 1.2],
[2019, 1.8],
[2018, 2.4],
[2017, 2.1],
[2016, 1.3],
[2015, 0.1],
[2014, 1.6],
[2013, 1.5],
[2012, 2.1],
[2011, 3.2],
[2010, 1.6],
[2009, -0.4],
[2008, 3.8],
[2007, 2.8],
[2006, 3.2],
[2005, 3.4],
[2004, 2.7],
[2003, 2.3],
[2002, 1.6],
[2001, 2.8],
[2000, 3.4]
]

df_inflation = pd.DataFrame(df_inf

df_inflation['Year'] = pd.to_numer

df_inflation0122 = df_inflation[(c

print(df_inflation0122)

#df_inflation0122.to_csv('inflatio

Year Ave
1 2022 8.0
2 2021 4.7
3 2020 1.2
4 2019 1.8
5 2018 2.4
6 2017 2.1
7 2016 1.3
8 2015 0.1
9 2014 1.6
10 2013 1.5
11 2012 2.1
12 2011 3.2
13 2010 1.6
14 2009 -0.4
15 2008 3.8
16 2007 2.8
17 2006 3.2
18 2005 3.4
19 2004 2.7
20 2003 2.3
21 2002 1.6
22 2001 2.8
```

```
In [113... # crimes from 2001 -2022

crime_data_large = pd.read_csv("C:

crime_data_large['Date'] = pd.to_c
```

```
<class 'pandas.core.frame.DataFra
me'>
RangeIndex: 7681524 entries, 0 to
7681523
Data columns (total 22 columns):
# Column Dtype
---
0 Primary Type object
1 Year int64
dtypes: int64(1), object(1)
memory usage: 117.2+ MB
Primary Type 0
Year 0
dtype: int64
```

```
In [116... #crimes_0122.to_csv('crimes_0122.c
```

```
In [117... import pandas as pd

crimes_0122 = pd.read_csv('crimes_
```

```
In [118... years = crimes_0122['Year'].unique()
print(years)
```

```
[2015 2018 2016 2014 2001 2020 20
19 2021 2012 2017 2013 2011 2007
2003
2010 2008 2009 2022 2002 2005 20
06 2004]
```

```
In [119... def categorize_crime(crime_type):

    economically_motivated = ['THE
                                'DE
    violent = ['BATTERY', 'ASSAULT
              'HOMICIDE', 'KIDNAP
    drug_related = ['NARCOTICS', '
    public_crime = ['LIQUOR LAW VI
                  'INTERFERENCE
                  'OBSCENITY', '
    property_related = ['CRIMINAL
    vulnerable_populations = ['OFF
    miscellaneous = ['OTHER OFFENS
                      'NON-CRIMINAL

    if crime_type in economically_
        return "Economically Motiv
    elif crime_type in violent:
        return "Violent"
    elif crime_type in drug_related
        return "Drug-Related"
    elif crime_type in public_crime
        return "Public Crime"
    elif crime_type in property_re
        return "Property-Related"
    elif crime_type in vulnerable_
        return "Vulnerable Populat
    elif crime_type in miscellanea
        return "Miscellaneous"
    else:
        return "Other"

crimes_0122['Category'] = crimes_6
```

```
crimes_0122.head(3)
crimes_0122.tail(3)

Out[119]:
```

	Primary Type	Year	Category
7681521	THEFT	2022	Economically Motivated
7681522	NARCOTICS	2022	Drug-Related
7681523	WEAPONS VIOLATION	2022	Public Crime

◀ ▶

```
In [120]: crimes_0122_cat = crimes_0122.drop('Year', axis=1)
crimes_0122_cat['Year'] = crimes_0122_cat['Year']
crimes_0122_cat.head()
```

```
Out[120]:
```

	Year	Category
0	2015	Violent
1	2015	Economically Motivated
2	2018	Economically Motivated
3	2015	Drug-Related
4	2015	Violent

```
In [121]: #crimes_0122_cat.to_csv('crimes_0122_cat.csv')

In [122]: crimes_0122_cat['Count'] = 1
grouped3 = crimes_0122_cat.groupby('Year')
crime_counts0122 = grouped3.pivot_table(index='Year', columns='Category', values='Count')
crime_counts0122.columns = ['Number of Property-Related Crimes', 'Number of Public Crime Crimes', 'Number of Economically Motivated Crimes', 'Number of Drug-Related Crimes', 'Number of Violent Crimes']
crime_counts0122.reset_index(inplace=True)
print(crime_counts0122)
```

```
28030
5
171084
27108
6
166415
26864
7
172148
26533
8
158626
25602
9
152985
22013
10
151526
20195
11
145439
17491
12
130350
18005
13
114210
17012
14
108746
17602
15
119977
17361
16
121640
17307
17
117754
17299
18
109242
16798
19
86899
2586
20
83617
3920
21
94628
2912

Number of Property-Related Crimes
Number of Public Crime Crimes
es \
0 6
9097 90
98
1 6
9821 85
47
2 6
9818 83
```

```
Year
Crimes \
0 2001
50573
1 2002
51794
2 2003
54290
3 2004
57069
4 2005
56243
5 2006
55822
6 2007
54466
7 2008
46519
8 2009
43549
9 2010
43400
10 2011
38610
11 2012
35495
12 2013
34132
13 2014
29128
14 2015
23944
15 2016
13337
16 2017
11693
17 2018
13583
18 2019
15070
19 2020
7495
20 2021
5009
21 2022
3530

Number of Economically Motivated Crimes
Number of Drug-Related Crimes
us Crimes \
0
194188
29670
1
189737
32602
2
185828
31150
3
181441
29533
4
171213

82
3
6
9077
83
31
4
7
1203
84
79
5
7
1629
88
03
6
6
7448
87
32
7
6
5152
83
99
8
5
8576
86
55
9
5
0054
88
17
10
4
5991
86
94
11
4
4071
87
58
12
3
8990
81
61
13
3
5337
78
76
14
3
5077
74
83
15
3
7324
63
17
16
3
5859
76
27
17
3
4731
86
53
18
3
3499
99
24
19
2
9054
107
09
20
2
8497
102
98
21
2
7960
93
63

Number of Violent Crimes
Number of Vulnerable Populations
Crimes
0 130972
2258
```


1 131705
2 123463
3 120871
4 115692
5 110946
6 110237
7 105750
8 95180
9 90596
10 84521
11 82686
12 75329
13 69647
14 69411
15 73002
16 72482
17 74273
18 74196
19 63231
20 64739
21 59300
1717

In [123... #crime_counts0122.to_csv('crime_cc

In [124... #2001 to 2022 Crime Rates

crime_counts0122 = pd.read_csv('cr

In [125... #2010-2022 Cost of Living

costs= pd.read_csv('col_and_cr_per

```
columns_to_drop = [  
    'Number of Drug-Related Crimes'  
    'Number of Economically Motiva'  
    'Number of Miscellaneous Crime'  
    'Number of Property-Related Cr'  
    'Number of Public Crime Crimes'  
    'Number of Violent Crimes',  
    'Number of Vulnerable Populati'  
]
```

```
for j in range(i + 1):  
    if not pd.isna(costsadd['Year'].iloc[j]):  
        found = True  
        next_year = costsadd['Year'].iloc[j + 1]  
        next_value = costsadd['Average Monthly Net Salary (After Tax)'].iloc[j + 1]  
        break  
  
    if found:  
        # Backward fill  
        for k in range(j, i):  
            curr_year = costsadd['Year'].iloc[k]  
            if curr_year == next_year:  
                costsadd['Average Monthly Net Salary (After Tax)'].iloc[k] = next_value  
                next_value = costsadd['Average Monthly Net Salary (After Tax)'].iloc[k + 1]  
            else:  
                # If no future data, fill with last known value  
                for k in range(j, i):  
                    if not pd.isna(costsadd['Average Monthly Net Salary (After Tax)'].iloc[k]):  
                        costsadd['Average Monthly Net Salary (After Tax)'].iloc[k] = costsadd['Average Monthly Net Salary (After Tax)'].iloc[k + 1]  
                        break
```

In [129... costsadd.head()

Out[129]:

	Year	Average Monthly Net Salary (After Tax)	Mortgage Interest Rate in Percentages (%), Yearly, for 20 Years Fixed-Rate	Average Rent Price
0	2001	2235.883433	4.577463	1057
1	2002	2271.657568	4.650703	1074
2	2003	2323.905692	4.757669	1099
3	2004	2386.651145	4.886126	1128
4	2005	2467.797284	5.052254	1167

In [130... cost_of_living0122est = costsadd

In [131... #cost_of_living0122est.to_csv('cos

In [132... cost_of_living0122est

```
costs.drop(columns=columns_to_drop,  
costs.head()
```

Out[125]:

	Year	Average Monthly Net Salary (After Tax)	Mortgage Interest Rate in Percentages (%), Yearly, for 20 Years Fixed-Rate	Average Rent Price
0	2010	2750.00	5.63	1300.87
1	2011	2800.00	5.26	1574.48
2	2012	3102.86	4.65	1725.11
3	2013	3686.99	4.37	1864.27
4	2014	3246.85	4.40	1819.54

In [126... #costs.to_csv('costs.csv', index=f

In [127... # Create a DataFrame with years 2001 to 2009
years_2001_to_2009 = pd.DataFrame(

Concatenate the original 'costs'
costsadd = pd.concat([years_2001_t

Display the modified 'costs' Dat
costsadd.head()

Out[127]:

	Year	Average Monthly Net Salary (After Tax)	Mortgage Interest Rate in Percentages (%), Yearly, for 20 Years Fixed-Rate	Average Rent Price
0	2001	NaN	NaN	NaN
1	2002	NaN	NaN	NaN
2	2003	NaN	NaN	NaN
3	2004	NaN	NaN	NaN
4	2005	NaN	NaN	NaN

In [128... import pandas as pd

```
inflation_dict0122 = df_inflation0122  
  
for column in costsadd.columns:  
    if column != 'Year':  
        for i in range(len(costsadd[  
            if pd.isna(costsadd[co  
                found = False
```

Out[132]:

	Year	Average Monthly Net Salary (After Tax)	Mortgage Interest Rate in Percentages (%), Yearly, for 20 Years Fixed-Rate	Average Rent Price
0	2001	2235.883433	4.577463	1057
1	2002	2271.657568	4.650703	1074
2	2003	2323.905692	4.757669	1099
3	2004	2386.651145	4.886126	1128
4	2005	2467.797284	5.052254	1167
5	2006	2546.766798	5.213926	1200
6	2007	2618.076268	5.359916	1230
7	2008	2717.563166	5.563593	1280
8	2009	2706.692913	5.541339	1280
9	2010	2750.000000	5.630000	1300
10	2011	2800.000000	5.260000	1574
11	2012	3102.860000	4.650000	1725
12	2013	3686.990000	4.370000	1864
13	2014	3246.850000	4.400000	1819
14	2015	3003.100000	4.180000	1880
15	2016	3578.850000	3.960000	2000
16	2017	3327.800000	3.950000	2020
17	2018	4055.270000	4.270000	1960
18	2019	4211.870000	4.270000	2270
19	2020	4541.450000	4.040000	2240
20	2021	5583.440000	2.960000	2080
21	2022	4976.190000	5.630000	2560

This data set, crime_rates_col_0122, has all of the data from 2001-2022 for the cost of living and crime rates in Chicago.

In [133... crime_rates_col_0122 = pd.merge(cc
crime_rates_col_0122.head()

Out[133]:

	Year	Average Monthly Net Salary (After Tax)	Mortgage Interest Rate in Percentages (%), Yearly, for 20 Years Fixed-Rate	Average Rent Price (per square meter)
0	2001	2235.883433	4.577463	1057
1	2002	2271.657568	4.650703	1074
2	2003	2323.905692	4.757669	1099
3	2004	2386.651145	4.886126	1128
4	2005	2467.797284	5.052254	1167

```
In [134]: #crime_rates_col_0122.to_csv('crime_rates_col_0122.csv')
```

Data Analysis on Estimated Cost of Living and Crime Rates from 2001-2022

Linear Regression 01-22

```
In [135]: crime_rates_col_0122 = pd.read_csv('crime_rates_col_0122.csv')
crime_rates_col_0122.set_index('Year', inplace=True)
```

```
In [136]: import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
import numpy as np

features = crime_rates_col_0122[['Average Monthly Net Salary',
                                'Mortgage Interest Rate in Percentages (%)',
                                'Average Rent Price',
                                'Average per Square Meter Built',
                                'Total Average Cost of all Utilities',
                                'Average Restaurant Meal Price',
                                'Average Drink Price',
                                'Total Average Cost of all Groceries',
                                'Average Fun Activity Price']]

targets = crime_rates_col_0122[['Number of Drug-Related Crimes',
                                'Number of Economically Motivated Crimes',
                                'Number of Miscellaneous Crimes',
                                'Number of Property-Related Crimes',
                                'Number of Public Crime Crimes',
                                'Number of Violent Crimes',
                                'Number of Vulnerable Populations Crimes']]

train_set2 = crime_rates_col_0122[['Average Monthly Net Salary',
                                    'Mortgage Interest Rate in Percentages (%)',
                                    'Average Rent Price',
                                    'Average per Square Meter Built',
                                    'Total Average Cost of all Utilities',
                                    'Average Restaurant Meal Price',
                                    'Average Drink Price',
                                    'Total Average Cost of all Groceries',
                                    'Average Fun Activity Price']]

test_set2 = crime_rates_col_0122[['Year',
                                   'Number of Drug-Related Crimes',
                                   'Number of Economically Motivated Crimes',
                                   'Number of Miscellaneous Crimes',
                                   'Number of Property-Related Crimes',
                                   'Number of Public Crime Crimes',
                                   'Number of Violent Crimes',
                                   'Number of Vulnerable Populations Crimes']]
```

- 2010-2022: 1,739,101,280.20
 - Comparison: The MSE for 2001-2022 is greater.
- Number of Miscellaneous Crimes:
 - 2001-2022: 49,905,058.23
 - 2010-2022: 10,835,455.56
 - Comparison: The MSE for 2001-2022 is greater.
- Number of Property-Related Crimes:
 - 2001-2022: 151,999,297.91
 - 2010-2022: 53,791,482.48
 - Comparison: The MSE for 2001-2022 is greater.
- Number of Public Crime Crimes:
 - 2001-2022: 11,356,275.34
 - 2010-2022: 47,126,905.35
 - Comparison: The MSE for 2001-2022 is less.
- Number of Violent Crimes:
 - 2001-2022: 1,494,972,959.53
 - 2010-2022: 244,431,371.51
 - Comparison: The MSE for 2001-2022 is greater.
- Number of Vulnerable Populations Crimes:

```
features_train2 = train_set2[features]
features_test2 = test_set2[features]

targets_train2 = train_set2[targets]
targets_test2 = test_set2[targets]

model = LinearRegression()
model.fit(features_train2, targets_train2)

Y_pred = model.predict(features_test2)

for i, col in enumerate(targets_test2.columns):
    mse = mean_squared_error(targets_test2[col], Y_pred[col])
    print(f"Mean Squared Error for {col}: {mse}")
```

Mean Squared Error for Number of Drug-Related Crimes: 576742226.1049045
Mean Squared Error for Number of Economically Motivated Crimes: 2135057447.1231465
Mean Squared Error for Number of Miscellaneous Crimes: 49905058.229021035
Mean Squared Error for Number of Property-Related Crimes: 151999297.90834552
Mean Squared Error for Number of Public Crime Crimes: 11356275.344829094
Mean Squared Error for Number of Violent Crimes: 1494972959.528236
Mean Squared Error for Number of Vulnerable Populations Crimes: 136950.93047917756

MSE Comparison for 2001-2022 data and 2010-2022 data Linear Regression:

- Number of Drug-Related Crimes:
 - 2001-2022: 576,742,226.10
 - 2010-2022: 1,005,668,502.89
 - Comparison: The MSE for 2001-2022 is less.
- Number of Economically Motivated Crimes:
 - 2001-2022: 2,135,057,447.12

- 2001-2022: 136,950.93
- 2010-2022: 99,718.31
 - Comparison: The MSE for 2001-2022 is greater.

The MSEs for the years 2001-2022 are lower for the "Number of Drug-Related Crimes" and the "Number of Public Crime Crimes". However, for all other crime categories, the MSEs are higher for the years 2001-2022 compared to 2010-2022. This suggests that the model's predictive accuracy varies depending on the crime category and the time period covered by the data.

```
In [137]: #Gradient Boosting Model with Logistic Regression

from sklearn.ensemble import GradientBoostingRegressor
from sklearn.metrics import mean_squared_error

#cost of Living features
features = [
    'Average Monthly Net Salary',
    'Mortgage Interest Rate in Percentages (%)',
    'Average Rent Price',
    'Average per Square Meter Built',
    'Total Average Cost of all Utilities',
    'Average Restaurant Meal Price',
    'Average Drink Price',
    'Total Average Cost of all Groceries',
    'Average Fun Activity Price'
]

# crime rate targets
target = [
    'Number of Drug-Related Crimes',
    'Number of Economically Motivated Crimes',
    'Number of Miscellaneous Crimes',
    'Number of Property-Related Crimes',
    'Number of Public Crime Crimes',
    'Number of Violent Crimes',
    'Number of Vulnerable Populations Crimes'
]

train_set = crime_rates_col_0122[features + target]
test_set = crime_rates_col_0122[features + target]
```

```
X_train = train_set[features]
y_train = train_set[target]
X_test = test_set[features]
y_test = test_set[target]

gbr_models = {}
mse_results_new = {}

for column in target:
    gbr = GradientBoostingRegressor()
    gbr.fit(X_train, y_train[column])
    y_pred = gbr.predict(X_test)
    mse_results_new[column] = mean_squared_error(y_test, y_pred)
    gbr_models[column] = gbr

# MSE
for crime_type, mse in mse_results_new.items():
    print(f"Mean Squared Error for {crime_type}: {mse}")
```

Mean Squared Error for Number of Drug-Related Crimes (new split): 62720813.46451841
Mean Squared Error for Number of Economically Motivated Crimes (new split): 272286099.66170764
Mean Squared Error for Number of Miscellaneous Crimes (new split): 7673964.064046727
Mean Squared Error for Number of Property-Related Crimes (new split): 18339833.941183243
Mean Squared Error for Number of Public Crime Crimes (new split): 336579.57022013585
Mean Squared Error for Number of Violent Crimes (new split): 114552166.52928697
Mean Squared Error for Number of Vulnerable Populations Crimes (new split): 364163.6898315363

MSE Comparison for 2001-2022 data and 2010-2022 data Gradient Boosting:

- Number of Drug-Related Crimes:
 - 2001-2022: 62,720,813.46
 - 2010-2022: 83,982,538.58
 - Comparison: The MSE for 2001-2022 is less.
 - 2010-2022: 4,817,044.99
 - Comparison: The MSE for 2001-2022 is greater.
- Number of Vulnerable Populations Crimes:
 - 2001-2022: 364,163.69
 - 2010-2022: 31,204.49
 - Comparison: The MSE for 2001-2022 is greater.

The MSE for "Number of Drug-Related Crimes" is lower for the period 2001-2022 compared to 2010-2022. However, for all other crime categories, the MSEs are higher for the period 2001-2022 than for 2010-2022. This indicates that the model performed better for the latter period across most crime categories, except for drug-related crimes.

8.1 Theoretical Data Model Conclusions

The utilization of a theoretical method (inflation rate backfill) to fill in the 2001-2009 data for this study, if not entirely accurate, may have significantly impacted the reliability and performance of the predictive

- Number of Economically Motivated Crimes:
 - 2001-2022: 272,286,099.66
 - 2010-2022: 265,574,084.30
 - Comparison: The MSE for 2001-2022 is greater.
- Number of Miscellaneous Crimes:
 - 2001-2022: 7,673,964.06
 - 2010-2022: 186,326.56
 - Comparison: The MSE for 2001-2022 is greater.
- Number of Property-Related Crimes:
 - 2001-2022: 18,339,833.94
 - 2010-2022: 8,181,936.31
 - Comparison: The MSE for 2001-2022 is greater.
- Number of Public Crime Crimes:
 - 2001-2022: 336,579.57
 - 2010-2022: 65,402.01
 - Comparison: The MSE for 2001-2022 is greater.
- Number of Violent Crimes:
 - 2001-2022: 114,552,166.53

model. Such an approach can compromise data integrity, as the filled-in data might not accurately reflect the real-world socioeconomic conditions during that period. Consequently, this lead to the model learning from data that fails to represent the true historical context, thereby diminishing its predictive accuracy. The observed higher Mean Squared Errors (MSEs) for this timeframe might be indicative of the model's struggle to discern meaningful patterns from potentially distorted or unrepresentative data.

This situation highlights the critical need for robust data imputation methods and caution in data interpretation and application. It's crucial to reevaluate the model's performance with more accurate and comprehensive data when available, and to apply advanced statistical techniques or leverage auxiliary data sources for more accurate data imputation. Until such improvements are made, the findings and predictions of the model should be treated with caution, acknowledging the limitations and potential

inaccuracies of the small data set.

9. References

Becker, G.S. (1968) 'Crime and punishment: An economic approach', *The Economic Dimensions of Crime*, pp. 13–68. doi:10.1007/978-1-349-62853-7_2.

Carteron, N. (2020) America in 2020 feels like France in 1788, Medium. Available at: <https://medium.com/lessons-from-history/america-in-2020-feels-like-france-in-1788-d4f2db892d5a> (Accessed: 05 December 2023).

Chuton, J. (2023) Cost of living 1980 vs 2023: Head to head comparison - HMH, Cost of living 1980 vs 2023: head to head comparison - HMH. Available at: <https://howmuchhub.com/cost-of-living-1980/> (Accessed: 05 December 2023).

Field, S. (1990) 'Trends in Crime and Their Interpretation: A Study of Recorded Crime in Post-War England and Wales', Home Office Research Study No. 119, London: Home Office.

Fox, C. et al. (2011) 'How will the recession affect crime rates in Greater Manchester?', *Safer Communities*, 10(3), pp. 17–30. doi:10.1108/17578041111171050.

Fuscaldo, D. (2022) How to adjust fixed income spending

Waley, A. (2011) *The analects of confucius*. London: Routledge.

Weatherburn, D. (2001) 'Contemporary Issues in Crime and Justice ', in *What causes crime?* Sydney: NSW Bureau of Crime Statistics and Research.

Resources

- Web Scraping:
 - Data Programming Lectures, Labs, and Videos: Sean McGrath, Goldsmiths University of London
 - Web scraping a table using BeautifulSoup tutorial, <https://www.youtube.com/watch?v=T1qv3ksMDq4>
 - One on One tutoring sessions: Brian DeWeese, Senior Software Engineer at Cboe Global Markets
- Data Cleaning and Processing:
 - Lectures, Labs, and Videos: Sean McGrath, Goldsmiths University of London
 - Data cleaning tutorial using Python, <https://www.youtube.com/watch?v=qxpKCBV60U4>
 - Data cleaning tutorial using Python, <https://www.youtube.com/watch?v=4yl3vVe0Jos>
- Exploratory Analysis:
 - Data Visualisation Lectures, Labs, and Videos: Joy Eze,

for inflation, AARP. Available at: <https://www.aarp.org/retirement/p-for-retirement/info-2022/inflation-adjusted-living-expenses.html> (Accessed: 02 November 2023).

Kang, S. (2016) 'Inequality and crime revisited: Effects of local inequality and economic segregation on crime', *Journal of Population Economics*, 29(2), pp. 593–626. doi:10.1007/s00148-015-0579-3.

Nicholson, K. (2022) Has the cost of living crisis had an impact on crime levels?, HuffPost UK. Available at: https://www.huffingtonpost.co.uk/of-living-crisis-impact-on-crime_uk_636a2b7be4b05f221e7d (Accessed: 02 November 2023).

Nunley, J.M., Stern, M.L., Seals, R.A. and Zietz, J. (2016) 'The impact of inflation on property crime', *Contemporary Economic Policy*, 34, pp. 483–499.

Rosenfeld, R. and Levin, A. (2016) 'Acquisitive crime and inflation in the United States: 1960–2012', *Journal of Quantitative Criminology*, 32, pp. 427–447.

Rosenfeld, R., Vogel, M. and McCuddy, T. (2018) 'Crime and Inflation in U. S. cities', *Journal of Quantitative Criminology*, 35(1), pp. 195–210. doi:10.1007/s10940-018-9377-x.

Goldsmiths University of London

- One on One tutoring sessions: Brian DeWeese, Senior Software Engineer at Cboe Global Markets